



M363 Unit 10
UNDERGRADUATE COMPUTING

Software engineering with objects



Components, patterns
and architecture

Unit **10**

This publication forms part of an Open University course M363 *Software engineering with objects*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

Java and all Java-based trademarks are trademarks of Sun Microsystems, Inc; Motif is a registered trademark of the Open Group; Rational Unified Process is a registered trademark of International Business Machines Corporation; Unified Modeling Language and UML are trademarks of Object Management Group, Inc. Design by Contract is a trademark of Interactive Software Engineering. Other product and company names may appear in the M363 course material. Rather than use a trademark symbol with every occurrence of a trademarked name, we use the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

Copyright © 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by SR Nova Pvt. Ltd, Bangalore, India.

Printed in the United Kingdom by Thanet Press Ltd, Margate.

ISBN 978 0 7492 1616 0

1.1



1	Introduction	5
2	Architectural design patterns and product lines	6
2.1	Software architectures	6
2.2	Frameworks	11
2.3	Product lines	13
2.4	Summary of section	16
3	Design patterns	17
3.1	Patterns for application layering	17
3.2	Singletons and Factories	24
3.3	Summary of section	33
4	Enterprise components and architectures	34
4.1	Component architectures	37
4.2	Service-oriented architectures	41
4.3	Summary of section	43
5	Summary	44
	References	45
	Acknowledgements	46
	Index	47

M363 COURSE TEAM

Robin Laney, Course Chair, Author and Academic Editor

Leonor Barroca, Author

Ralph Greenwell, Course Manager

Charles Haley, Author

Nigel Kermode, Critical Reader

Martin Shepperd, External Assessor, Brunel University

Richard Walker, Critical Reader

Andy Allum, Course Website Developer

Kim Dulson, Software Procurement

Phillip Howe, Media Assistant

Callum Lester, Software Developer

Tara Marshall, Print Procurement

Sandy Nicholson, Freelance Copy Editor

Andy Seddon, Media Project Manager

Lucinda Simpson, Editor

Sue Stavert, Technical Tester

Andrew Whitehead, Graphic Artist

Thanks are also due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1

Introduction

In this unit, we explore how the development of software can be made more productive and larger problems tackled through the reuse of experience gained and code produced on previous projects. This experience has been captured in *patterns* and *component architectures*. We look first at patterns in Sections 2 and 3, and then component architectures in Section 4.

In *Unit 6*, you learned how to build design models based on artefacts, developed during analysis, by applying GRASP for assigning responsibilities to classes during design. In Sections 2 and 3 of this unit, you will learn how to apply other types of patterns to solve a variety of design problems. In Section 2 we look at patterns at an architectural level. We begin with the Layers pattern, an architectural pattern that is particularly relevant in the construction of enterprise software systems. We also look briefly at the notions of frameworks and product lines, concepts that can have profound implications on a design. In Section 3 we introduce the Model–View–Controller, the Application Controller and Observer patterns.

In Section 3 we will also look at two well known design patterns, Singleton and Factory. These are among the first design patterns ever codified, and are commonly adopted in object creation. We discuss their relationship to the GRASP Creator pattern that you studied in *Unit 6*.

Section 4 extends the discussion of components in Unit 9 by introducing component-based development using Enterprise JavaBeans. A component architecture and framework, Java Platform, Enterprise Edition, is presented. The section closes with a brief discussion of service-oriented architectures.

2

Architectural design patterns and product lines

Best practice in design has been codified in the form of architectural and design patterns. **Architectural patterns** (also known as architectural styles, see Shaw and Garlan (1996) and Bass et al. (2003)) codify recurrent software architectures, by describing the key elements of the architecture and how they fit together.

2.1 Software architectures

The SEI's website is http://www.sei.cmu.edu/architecture/published_definitions.html (Accessed December 2007).

At Carnegie Mellon's Software Engineering Institute (SEI), there is a record of various definitions of software architecture that have appeared in the literature. The catalogue is publicly available from their website: to date, it includes some 30 definitions.

Although a standard definition is lacking, there is a common agreement that a software architecture refers to the broad structure of a software system. It describes its major parts, and how they are put together and interact. It also captures an underlying rationale – why the system is structured in that particular way. In a sense, a software architecture is perceived as that part of a system design that has to be established early on in development. It can be thought of as a foundation for the subsequent detailed design, and one that, once established, should not be modified substantially.

Software architectures, as other parts of a system, can be described using models. These models may encompass a number of **architectural views** of the system, which collectively describe the whole system. Different views are useful in separating concerns of different stakeholders of the system. For instance, Kruchten (1999) identifies, among others, the following views.

- ▶ The **functional (or logical view)**, which describes the system's main functional elements, their responsibilities, interfaces and primary interactions. A functional view is the cornerstone of most architectural descriptions.
- ▶ The **process view**, which addresses concurrent aspects of the system at run-time, including all system processes and system start-up and shut-down.
- ▶ The **deployment view**, which describes the environment into which the system will be deployed, including processing nodes, network interconnections and disk storage facilities required; it maps the software elements onto the run-time environment that will execute them.

In this course we are primarily concerned with the functional view of a system. Therefore we will use the term *system architecture* to indicate the functional view of the architecture of a system. You should be aware, however, that this is only a small part of the picture.

The Layers pattern

In this subsection we describe a widely adopted architectural pattern: the **Layers pattern**. The following description is based on that of Buschmann et al. (1996).

- ▶ *Name* Layers.
- ▶ *Intent* To help structure the architecture of a system into groups of subfunctions, each at a particular level of abstraction.

- ▶ *How it works* The system is structured into an appropriate number of layers, which are then (logically) placed on top of each other, with each layer making use only of services provided by the layers below it.
- ▶ *When to use it* This pattern applies in the development of large complex systems, which require decomposition, in particular when there is a mix of low- and high-level issues to be addressed.
- ▶ *Example* Networking protocols (e.g. the ISO OSI seven-layer model) are probably the best known examples of layered architectures. Such protocols usually consist of a set of rules and conventions that describe how computers communicate across machine boundaries and over networks. They are usually organised in subprotocols at various levels of abstraction, ranging from the physical layer (dealing with transferring electrical signals on a physical network) up to the application layer (dealing with transferring data between remote applications).

The Layers pattern has a number of advantages, including:

- ▶ separation of concerns: each layer addresses a particular concern, so that all the concerns are kept separate;
- ▶ lowered coupling: the dependency between layers is kept low by following clearly stated rules, typically from high- to low-level layers;
- ▶ reuse: general services (for instance access to remote devices over a network) are kept separate from specific application logic and hence are potentially reusable;
- ▶ flexibility: because of low coupling and separation of concerns, changes in elements of the system are localised and hence easier to achieve; entire layers may be replaced with alternative implementations of the same basic services.

There are some disadvantages too:

- ▶ replication: there may be a need to replicate information across layers;
- ▶ performance: there may be a need to perform extra transformations at each layer.

SAQ 1

- (a) What are the main issues addressed by the Layers pattern and what are the main mechanisms used to address them?
- (b) How does the adoption of the pattern facilitate reuse and flexibility?

ANSWER.....

- (a) The main issues addressed by the pattern are achieving low coupling between elements, and separating concerns. This is realised by organising the system elements in layers, each layer addressing a particular concern, and managing the coupling between layers carefully.
 - (b) The implementation of general services can be separated from that of other parts of the system. Hence services can be potentially reused in other applications, or their implementation replaced while still providing the same basic services.
-

The Layers pattern has been widely applied to enterprise systems. These are systems characterised by large quantities of data, which are usually *persistent*, that is, they need to be kept for several runs of the system, typically several years. Such systems are often used by many users simultaneously. Different types of user may be using the system to accomplish disparate tasks, which require many diverse user interfaces. Enterprise systems usually integrate with other systems in order to support some business within an enterprise, and they are often governed by complex business rules.

A relational database is one example of persistent storage.

Enterprise systems tend to be large and complex, with a mix of concerns, some of which could be considered as high-level (for instance, user interaction) and some as low-level (for instance, interfacing with other devices). The Layers pattern can be regarded as a way of managing the complexity of enterprise systems, by organising the large-scale functional structure of the system into discrete layers of distinct, related responsibilities, with a clear separation of concerns between the layers.

From a design viewpoint, the main difficulty in applying the Layers pattern is to decide how the system should be separated into layers and how to assign responsibility to each layer. Many flavours of the Layers patterns for enterprise systems have appeared in the literature. The basic one includes three principal layers.

- ▶ The *presentation layer*, which handles the interaction between the users and the system; it captures user events and includes the system's user interface.
- ▶ The *domain (or business) layer*, which defines the work the system was designed to do; it contains domain logic and implements domain rules.
- ▶ The *data source layer*, which handles the storage of persistent data, and networking with other systems.

The term *domain* is used here to represent a layer in the architecture of an enterprise system which should contain the design related to the business logic of the application. It should not be confused with domain modelling, which is the activity concerned with building models of a domain, independently of any software solution.

Other flavours of the Layers pattern introduce mediating layers between the principal layers described above. A common variant is the five-layer pattern. In this variant, there are two extra layers.

- ▶ The *application layer*, which mediates between the presentation and domain layers; it may maintain session states, or perform transformations to consolidate disparate data for presentation.
- ▶ The *technical services layer*, which mediates between the domain and data source layers. It may contain services for persistence, transactions, load balancing, or security; or interfaces to external systems.

Note that when the Layers pattern is applied to enterprise systems, the emphasis is more on decomposing an application into subsystems from general to more specific, rather than from high- to low-level of abstraction. For instance, there is no reason to consider a domain layer as a lower level of abstraction than the presentation layer, although the presentation layer is more application-specific than the domain layer, which could be shared among many applications (each with their own bespoke presentation layer). Also, the strict dependency rule from higher to lower layers does not always apply: for instance, presentation and domain layers tend to depend on each other because the presentation layer must be able to display information from the domain layer to the user, as well as conveying information, which is derived from the environment, to the domain layer. Interlayer dependencies have to be handled carefully in order to keep the level of coupling within the system low.

Layers and tiers

The terms **layer** and **tier** are sometimes used interchangeably in the literature to refer to a layered architecture. The two terms, however, have slightly different connotations: a layer is a logical partitioning of a system, whilst a tier is a physical partitioning. For instance, a web browser connected to a remote site is a simple two-tier system, in which one tier is part of the presentation layer (a web browser) running on a client computer, and the other tier is the rest of the presentation layer plus the domain and data source layers running on a remote server. However, the application may well be organised as a 'fat client' two-tier application where the presentation and domain layers run on the client computer and the data source layer runs on a remote server, or as a three-tier architecture where the presentation layer is on the client, the domain layer runs on one server, and the data source layer runs on another server. In this course, we will avoid the term tier, and instead use layer whenever we refer to the logical architecture of a system.

We can represent a layered architecture using the UML notation for a package. A UML package, introduced in Section 6 of *Unit 7*, is a grouping of UML elements. All kinds of UML elements can be organised into packages, although they are most commonly used to group classes. The notation for a package is a rectangle with a small tab attached to its top left-hand side, as illustrated in Figure 1, where a UML package is defined to represent each layer in the two flavours of Layers pattern discussed above.

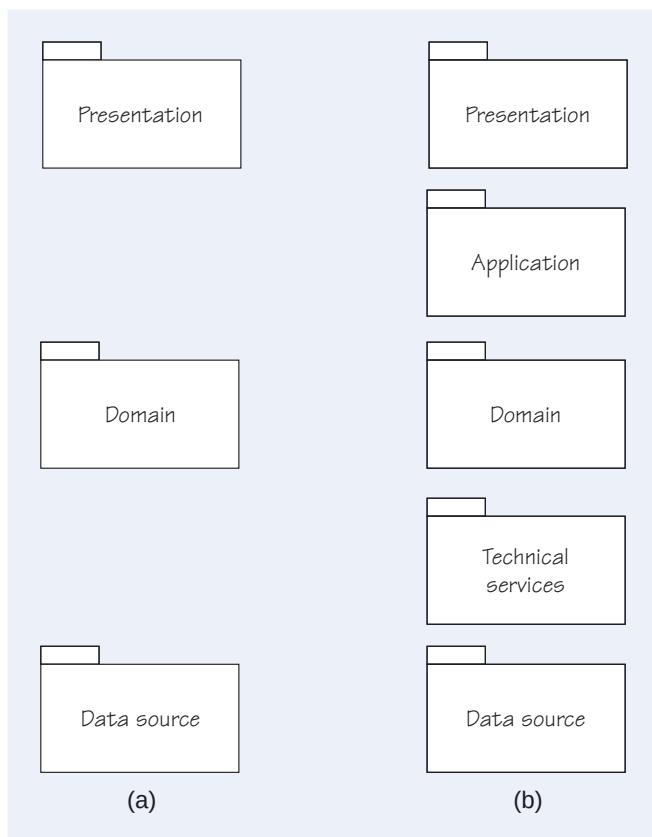


Figure 1 Two flavours of the Layers pattern, represented as UML packages

SAQ 2

- (a) Describe the relationship between the two styles of Layers pattern illustrated in Figure 1.
- (b) What do you think may determine the flavour of the patterns that should be adopted for the architecture of a particular enterprise system?

ANSWER.....

- (a) The five-layered version (Figure 1(b)) introduces two mediating layers between the fundamental layers of the three-layered version (Figure 1(a)).
- (b) Complexity and flexibility are determining factors. For instance, if the relationship between the presentation and domain layers is straightforward and can be easily modified and tested, then it might be more efficient to distribute the logic between these two layers rather than introducing a third one. On the other hand, in a system that allows complex interactions to occur with a large variety of users and where changes are frequent, the introduction of an application layer might allow the logic in the two other layers to be independently modified, tested and validated. A similar argument applies to the technical services layer.

If we look at a system from the viewpoint of a layered architecture, all the design issues we have addressed so far relate to the design of the domain layer. Later in the course we shall address design issues in other layers. Figure 2 shows a layered view of the design for the hotel system using a UML package. The *HotelChainBusiness* package represents the design of the domain layer of the system and contains the whole class diagram for the hotel system developed so far. The package name is indicated in the tab of the package symbol. The figure is based on the class diagram for the hotel system produced at the end of *Unit 8*.

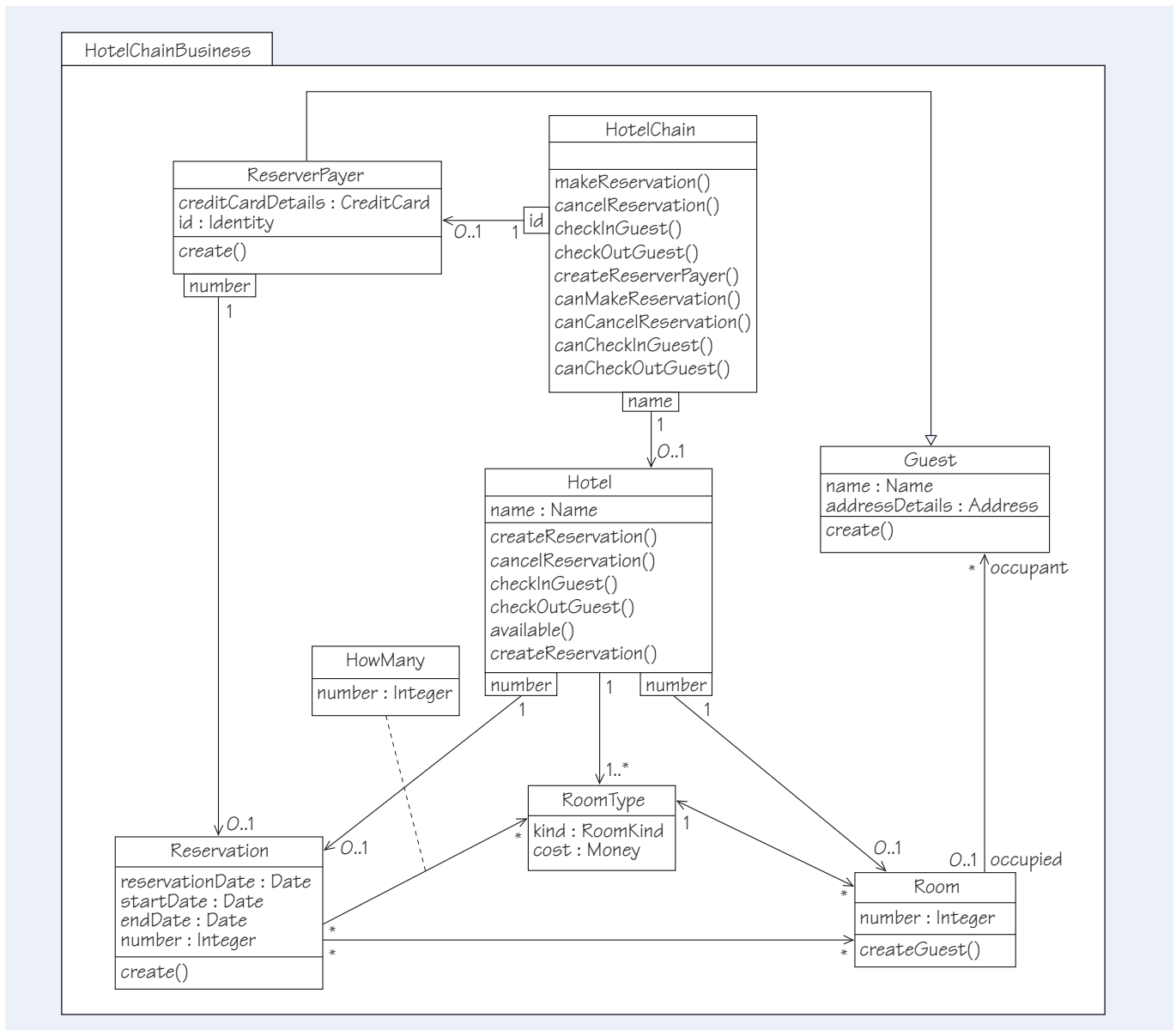


Figure 2 Domain layer package for the hotel system in a Layers architecture

2.2 Frameworks

When developing a new system, we may notice that at a high level of abstraction, aspects of our new system are similar to aspects of another system that we've encountered before. That is to say, the architectural models of the two systems have aspects in common. We might therefore save some effort by reusing part of the architectural model of the previous system, along with the code that is common.

Chunks of architecture and code that can be reused in this way are usually called **frameworks**. Although the framework contains some code for the framework itself, most of the code would be for components that could be slotted into the framework. These components would be stored in a component repository. Each framework would be accompanied by suitable documentation to facilitate its use.

Consider software to control a heating system for a building. Heating systems always consist of essentially the same elements:

- ▶ a heater;
- ▶ a thermostat, which switches the heater on and off depending upon the temperature in the building;
- ▶ a control to set the desired temperature;
- ▶ an optional timing device to determine when the heating should switch on and off;
- ▶ an optional override switch to set whether the heating should be always on, always off, or subject to the timer.

Each of these physical elements, apart from the heater, is an actor in the system, leading to the use case diagram of Figure 3.

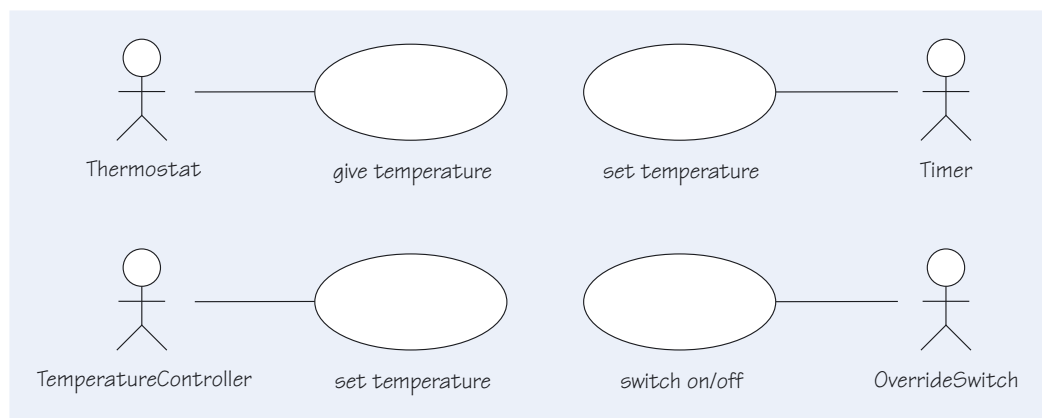


Figure 3 Use case diagram for a heating control system

Each device can be modelled by an abstract class, and the key component of the architectural model will be the class diagram showing how these interact.

In Figure 4, several sensing devices have been shown, all of which send signals to the monitor about the environment. The monitor compares the signals with the schedule (a list of times at which the heating is to be on or off). The abstract class *Monitor* allows for different kinds of heaters (for example, oil, gas, or electricity). The abstract class *Regulator* represents the controls on the heater, which generally depend on the kind of heater.

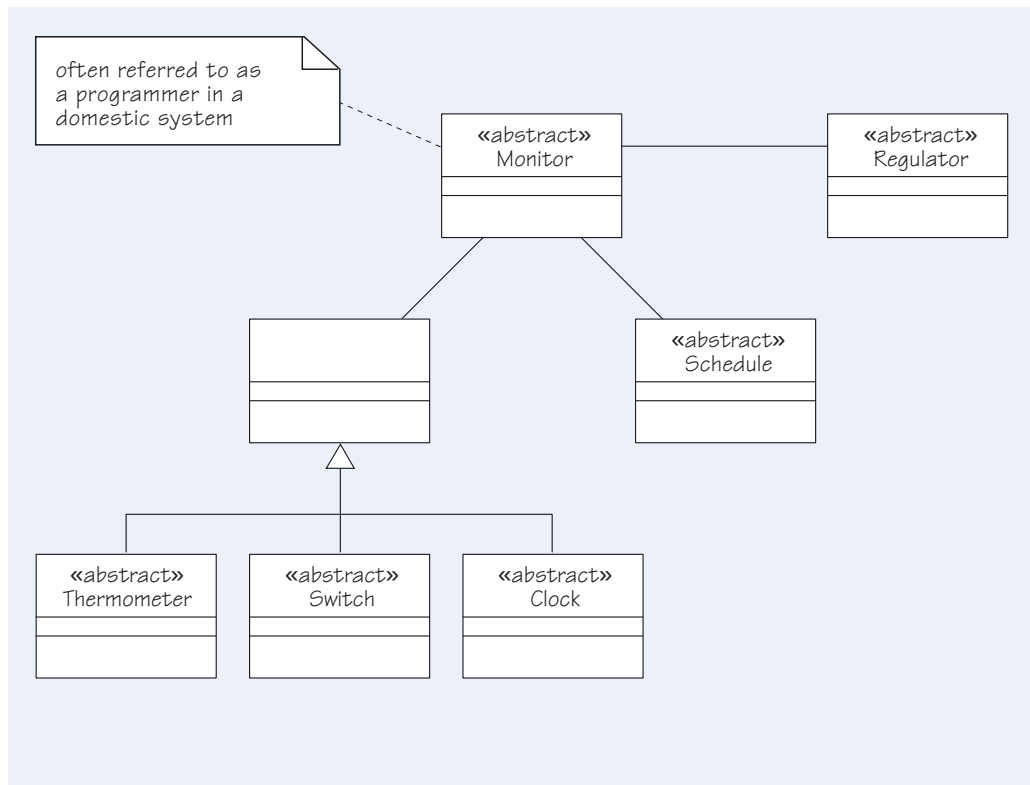


Figure 4 Class diagram for a heating control system

In addition to the framework in Figure 3 and Figure 4, there could be a number of classes supplied that implement the abstract classes and that could be substituted for the abstract classes of the framework. For example, there could be a number of different concrete *Monitor* classes, each of which would implement different monitoring policies.

We could use this framework in a number of different heating control systems, either using the architecture to guide us, or following the class diagram very closely and merely substituting concrete classes for the abstract classes.

This discussion could give the impression that it is easy to generate a framework. This is not the case. Frameworks, such as the ones presented in this unit, were developed over many years of experimentation and industrial experience, and they depend upon a lot of supporting software.

2.3 Product lines

It frequently happens that a company wants to produce many similar software systems. For example, engines in motor cars are controlled by an on-board computer using quite sophisticated software. Different engines and cars require slightly different engine-control software, but engine-controllers are essentially the same. A manufacturer of engine-control software can produce a series of engine-controllers for different customers, reusing major parts of the software from customer to customer with only those parts peculiar to the particular customer being different. Using the jargon of engineering, this is known as a **product line**, a line of products that differ only in detail. Often, products in a product line are based on a common framework.

Developing product lines for software began in the mid-1980s, mostly in the defence sector where the costs and reliability of routine weapons' systems gave a lot of concern. Trials were made on rocket guidance systems. The methods moved from defence to the civil sector, and by the late 1990s the practice of product lines had become established for normal commercial systems as well.

The product-line process

The way to develop product lines is to build upon the idea of frameworks. We have already covered all the essential ingredients, so all that remains is to pull these together. There are a number of steps that are undertaken.

- 1 **Product-line initiation.** Before development of a product line begins, there will have been recognition that there is a class of commonly recurring software systems where solutions do not differ much from each other. This leads to domain analysis.
- 2 **Domain analysis.** Domain analysis is a form of requirements elicitation in which existing systems in the domain are examined and domain experts are consulted. The area in which the product line will operate is thoroughly analysed to obtain the general customer needs and the terminology that is normally used, leading on to general models of data and process.
- 3 **Architecture specification.** Further analysis then leads to the architecture for the product line. There may be several such *reference architectures* produced. These are further elaborations of the domain model to add detail, particularly about the prospective implementation, and with flexibility kept in mind. Each reference architecture will include any encoding to make the architecture work once components for deployment in the architecture have been decided upon.
- 4 **Component collection.** As with a framework, each architecture will be accompanied by a repository of candidate components that could be used in the architecture. Nevertheless, it must be recognised that complete coverage is neither possible nor desirable, and that some new components and glue code (code that adapts new components to the framework) may need to be written for a particular use of the architecture. All components in the repository must be appropriately documented.

These four steps, each very complex and difficult, leave us in a position to produce the software systems for which the product line was created. For this the following three further steps are required, each of which must be repeated for each new product in the line.

- 5 **Specific-requirements capture.** The specific requirements for a new product are captured, as specialisations and extensions of the domain model.
- 6 **Architecture specialisation.** A reference architecture is selected based on the specific requirements, and changes are made to incorporate the specialisations and extensions of the domain model corresponding to the new requirements.
- 7 **Component selection and specialisation.** The product that meets the new requirements is then built, selecting components from the repository where these exist, and adapting them as necessary. If there is nothing appropriate in the repository, a new component may be built by those managing the repository for this product line. Alternatively, you may develop the new component yourself, if it is so specialised as to be unlikely to be required in other products.

The final step is the following.

- 8 **Integration and release.** Integrating the components into the architecture should be relatively easy – if the architecture has been well designed!

All eight steps include the normal quality controls of review and testing to ensure that the result is satisfactory.

SAQ 3

- (a) What do we mean by software architecture?
 (b) How does an architectural pattern relate to software architecture?

ANSWER.....

- (a) Software architecture is a high-level abstract description of the structure of a software system – its major parts and how they are interrelated.
 (b) An architectural pattern focuses on particular elementary principles used in software architectures, so that aspects of the way a particular architecture works can be accounted for by reference to architectural patterns.

SAQ 4

- (a) What are the various ingredients that make up a framework?
 (b) How does a domain model differ from a conceptual model?
 (c) What is a common business object?
 (d) How do frameworks help in the reuse of components?

ANSWER.....

- (a) A framework comprises an architecture, together with a component repository that contains components suitable for use within the framework, plus whatever documentation would facilitate the use of the framework.
 (b) A domain model addresses a general set of possible problems or requirements, whilst a conceptual model addresses a specific problem.
 (c) A business object is any entity, such as a customer, that occurs in a business application. A common business object is one that recurs widely and often, and is worth placing in a component repository.
 (d) Components may not quite fit into a particular application, or can be too general to be useful. A framework only places in the repository those components that do fit and are useful. The framework will guide the user to the component required, so that general search methods are not required.

Exercise 1

Describe very briefly a method for developing a framework, based on the discussion of the Layers pattern and frameworks above.



Solution.....

- 1 Characterise the domain in general terms.
- 2 List the different elements of the system, possibly with the help of a use case diagram.
- 3 Develop a preliminary class diagram for the framework, with a just a few abstract classes, based on what your examples have in common.

- 4 Identify the layering styles that the framework is to use.
- 5 Identify some of the component concrete classes that could be substituted for the abstract classes.
- 6 Use this class diagram to modify and elaborate the class diagram of the framework.



Exercise 2

Suppose that your company makes a range of computer games, and now wants to offer simple variants on these games that are personalised to a company or a person. The rules may be changed in simple ways, and names and pictures may be changed to fit characters in the company's or the person's life, and so on. How would you design such a software product-line for your company?

Solution

An analysis should be made of a number of games, to determine the variety of customisations that could be made, referring to prospective customers and games experts wherever possible (domain analysis). Reference architectures would then be developed in which the games themselves were represented at a high-level, highlighting only those places where customisation may be required (architectural specification). A repertoire of possible customisations that people might want would be built up, such as variant game rules, or unusual card faces or different game characters, also allowing for words and images requested by the customer to be inserted (component collection).

2.4 Summary of section

In this section you saw how architectures can be used to guide software development, giving tried and tested high-level solutions to common problems. Reusable architectures, together with code including a repository of components that are known to be useful in that context, are called frameworks, and can form the basis for the systematic development of software systems in product lines.

3

Design patterns

A **design pattern** codifies a solution to a recurring design problem. This solution is rooted in practice, in that it codifies what experienced practitioners do when they encounter a particular software problem.

Since the *patterns movement* started in the mid-1990s, many pattern catalogues have been published. Some have focused on design; others have addressed different aspects of software development, from problem identification to writing code.

Pattern catalogues come in many flavours and apply within different activities of a development process. For instance, Jackson's *Problem Frames* (2001) is a catalogue of basic software problems; Fowler's *Analysis Patterns* (1997) is a catalogue of patterns which are usefully applied during analysis; *Design Patterns: Elements of Reusable Object-oriented Software* by the 'Gang of Four' (Gamma et al., 1995) is the most famous catalogue of design patterns; and Grand's *Java patterns* (1998, 1999) are **idioms**, that is, patterns which apply to a specific (programming) language.

In the rest of this section, you will study a number of design patterns that are frequently used in the design of enterprise systems. All the patterns you will learn combine well with the Layers pattern studied in Section 2, although they could equally well be adopted in other types of architecture.

There are a few things about design patterns of which you should be aware. Applying patterns may require some adaptation. It is unlikely you will encounter exactly the same problem twice; hence a previous solution may need changing in some way. Also, problems do not always match patterns exactly; hence some creativity may be required to adapt the proposed pattern solution to the given problem. Finally, although each pattern addresses a specific problem, patterns are not unrelated: some are alternative solutions to the same problem; others need to be combined to provide the required solution.

3.1 Patterns for application layering

From the discussion of the Layers pattern in Section 2, we can identify two separate concerns in the interaction between actor and system; these are:

- ▶ allowing the user and system to communicate, in order to exchange information and data;
- ▶ executing relevant domain logic, which allows, for instance, establishing whether certain conditions are satisfied and guiding further interaction between the actor and the system.

There are design patterns that help to address and separate these concerns when the interaction with actors takes place through user interfaces, and we will look at three of them: the **Model–View–Controller pattern**, the **Application Controller pattern**, and the **Observer pattern**.

Smalltalk is an object-oriented language introduced in the 1970s by Alan Kay's group at Xerox PARC, and the first object-oriented language to integrate graphical user interfaces (GUIs) and interactive program execution.

Model–View–Controller (MVC) pattern

The pattern in this subsection was first introduced in the late 1970s in Smalltalk. Many user interface component libraries today are based on MVC principles.

- ▶ *Name* Model–View–Controller (MVC).
- ▶ *Intent* To split user interface interaction into three distinct roles.
- ▶ *How it works* It identifies three roles. The first is the *model*, corresponding to an object with some information about the domain. The model contains data and behaviour and is not a visual object. In a layered architecture, the model resides in the business layer. The *view* is the graphical manifestation of the model in the user interface: it is what the user perceives of the model. Views reside in the presentation layer of a layered application. The *controller* handles all user inputs which affect the model. Controllers are also presentation-layer objects. The pattern works in such a way that user inputs to the controller translate into changes to the model's state, which in turn are reflected in the view. The view and the controller are both dependent on the model, as shown in Figure 5.

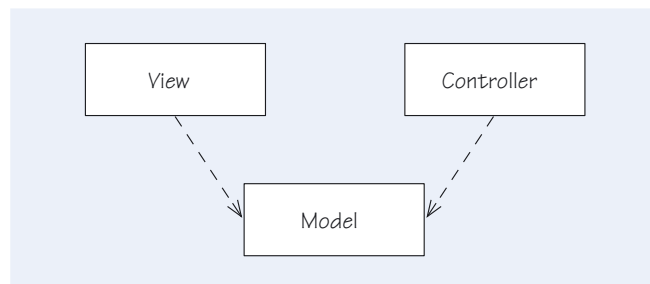


Figure 5 Dependencies in Model–View–Controller

- ▶ In many implementations of this pattern, the view and the controller are tightly coupled to a point that they are often integrated into the same object. This is the case, for instance, in the user interface (UI) components of Java's Swing library.
- ▶ *When to use it* Use this pattern to separate the presentation from the model. The advantages of this separation are many, including:
 - separation of concerns: when designing a model you focus on business issues; when you design a presentation you are concerned with aesthetic and usability issues;
 - ease of testing: domain logic is easier to test and tests can be automated; graphical interfaces are hard to test and tests are by and large manual;
 - flexibility: multiple presentations may be developed for the same model, allowing the design to cater for different types of users and different contexts of use.
- ▶ *Example* Figure 6 shows an example of a *Clock* (the view) showing the current time of *Timer* (the model), with a *Controller* which allows the user to set the current time to an arbitrary value.

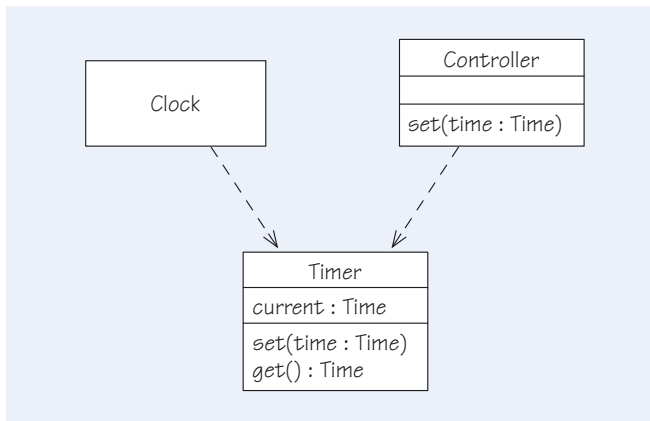


Figure 6 MVC structure for a clock system

Application Controller pattern

- ▶ *Name* Application Controller.
- ▶ *Intent* To have a centralised point for handling screen navigation and the flow of an application.
- ▶ *How it works* An application controller has two main responsibilities: deciding which domain logic to run and deciding on the view that will display the response. To do this the controller needs to be able to reference both presentation and domain objects, as indicated in Figure 7.

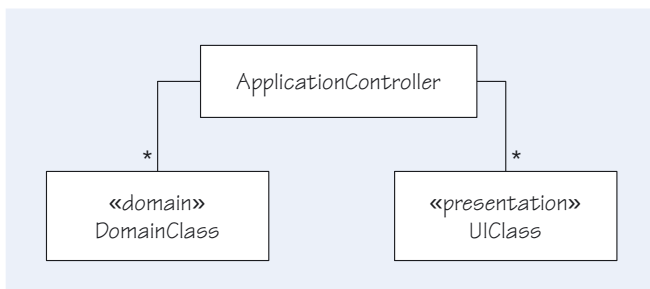


Figure 7 An application controller

- ▶ *When to use it* Use this pattern when the UI has many different screens which should be targeted by the user according to well defined rules, in particular when such rules depend on the current state of business objects.
- ▶ *Example* Figure 8 shows a possible interaction between an actor and the system via an application controller. The controller receives a request from the actor via some user interface class; it then issues some commands to a domain object in order to check whether a condition is satisfied and get some data; the data is then returned to the actor via the user interface. Note that this is only an illustration of a possible interaction. In a real system, a number of business objects are likely to be involved in the interaction with the application controller.

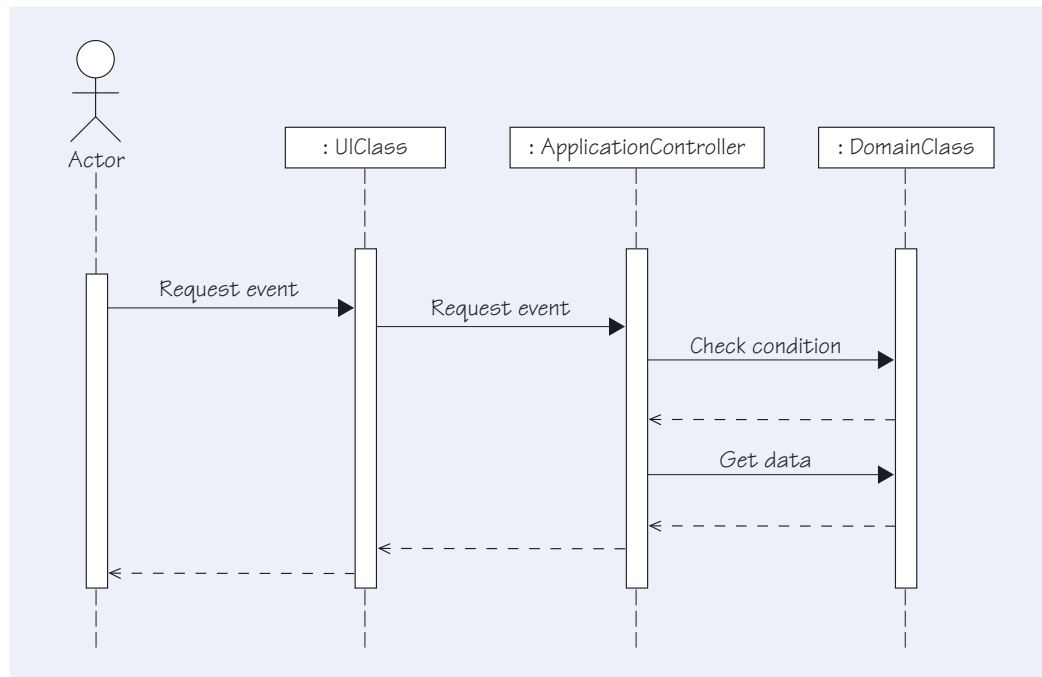


Figure 8 Sequence diagram for an application controller

SAQ 5

Discuss how the Application Controller pattern may apply in a layered architecture.

ANSWER.....

The Application Controller pattern fits nicely in a layered architecture, as illustrated in Figure 9. The controller may sit in the application layer and mediate between presentation and domain objects. Actor events are translated into system events between the interface and the controller; this may result in a number of messages between the controller and domain objects.

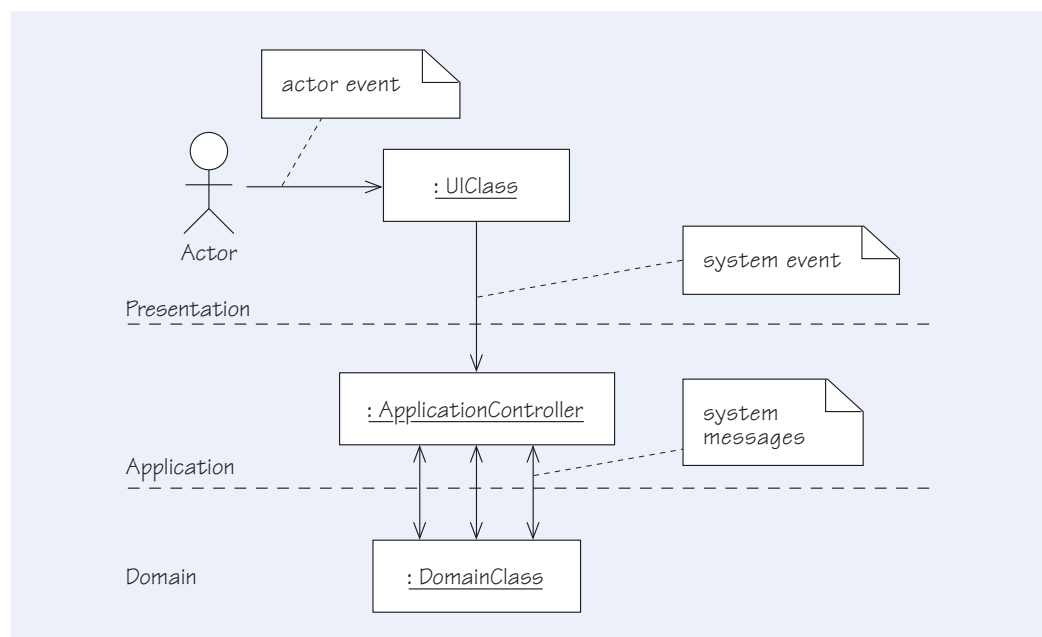


Figure 9 Layering of an application controller

SAQ 6

Discuss possible advantages of separating presentation from domain logic.

ANSWER.....

The two main advantages of this separation are: first, presentation logic is not affected by changes in the implementation of the business logic; second, business logic can be tested separately from the presentation logic.

Instances of an application controller mediate between instances of UI and domain classes. It is important, though, to distinguish the responsibilities of the UI classes from those of the controller. Among the responsibilities of the UI classes is that of checking the validity of all user inputs from a syntactic viewpoint: for instance, that an entered password has the correct number of characters, or that strings or numerical values are entered wherever they are required. The application controller has responsibility for the flow of control which will effect the execution of system operations, as well as coordinating system responses to the user via the UI classes. Table 1 gives a summary of the responsibilities of the UI classes and the application controller.

Table 1 Responsibilities of UI classes vs. the application controller

UI Classes	Application controller
Read user inputs Display system outputs Verify the syntax of user inputs	Direct the flow of control of a system operation Coordinate screen navigation in the UI Execute relevant business logic Map value and object identities

SAQ 7

What sort of business logic will be invoked by the application controller?

ANSWER.....

The application controller will invoke business logic to:

- ▶ locate class instances within the system, by mapping value and object identities;
- ▶ invoke methods on business objects to check operations' preconditions (the application controller takes the role of the client in the operation contract);
- ▶ delegate the satisfaction of the operations' postconditions to business objects.

SAQ 8

Discuss the differences and similarities between the application controller and the controller in the MVC pattern.

ANSWER.....

The application controller isolates and abstracts business logic from presentation logic, and controls the workflow of a business process. The MVC controller manages user inputs through a user interface in order to modify business data. Thus the contexts of their applications are different: a business process versus a user interface.

The main similarity between the two controllers is that they support a separation of concerns: presentation logic from business logic in the case of the application controller; data from the user's view in the case of the MVC controller.

Observer pattern

The Observer is a ubiquitous pattern which addresses a very general and recurrent problem in software: that of providing a loose dependency between objects. We introduce it here because it is often used within user interfaces.

- *Name* Observer (also known as Publisher–Subscriber).
- *Intent* To design a one-to-many dependency between objects so that when some internal state change occurs in one object, all its dependants are notified automatically.
- *How it works* The pattern realises a change-propagation mechanism in which one object has the role of the *subject* (or *publisher*), and its dependants the role of *observers* (or *subscribers*). The subject keeps a registry of observers, and notifies any change to all the observers in the registry. It is, however, the responsibility of the observers to register (or de-register) themselves from the registry. It is also the responsibility of the observers to take an appropriate action upon notification. There are two extreme variants of this pattern depending on whether the subject sends the observers detailed information about the change that has occurred. The *push model* is one in which the subject sends such information when notifying its observers. The *pull model* is one in which the subject sends the notification only and it is the responsibility of the observers to find out what has changed. The structure of the pattern is shown in Figure 10. Note that the concrete observer has to provide some implementation of the `update()` method which performs an appropriate action when a notification is received.

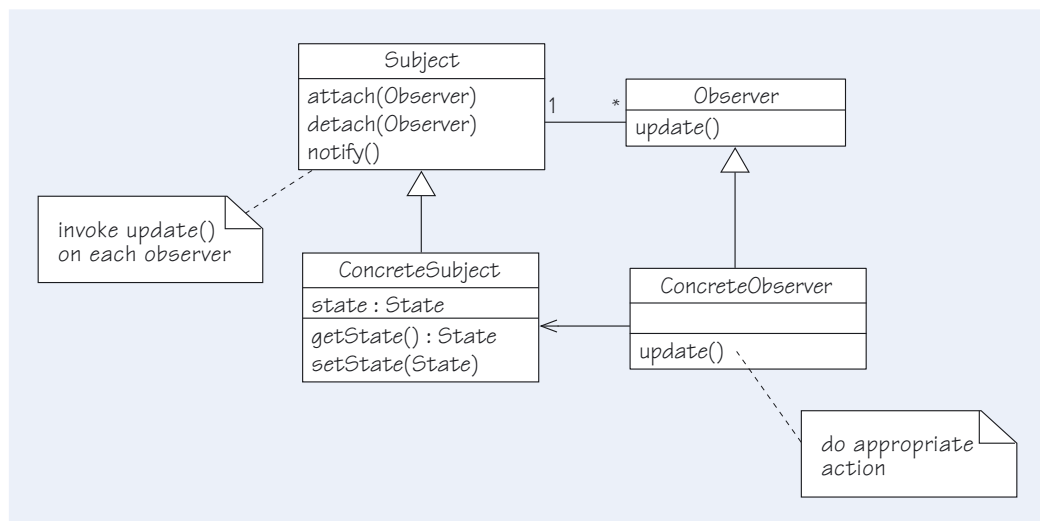


Figure 10 Example of the Observer pattern

You can find a Java example of the Observer pattern on the course website.

- *When to use it* Use this pattern when the number or the identity of the observers of an object is not known or may change over time, or when the object and its observers should not be tightly coupled.
- *Example* The relationship between the view and the model in an MVC design can be realised by applying the Observer pattern. The view would register with the model, and would be notified every time the model's state changed, so that it could reflect the changes in the UI.

SAQ 9

In this question we consider a variant of the heating system described in Subsection 2.2. The system, *HomeWarm*, is a central heating system for domestic use. It allows the user to set up a target home temperature to be maintained by the system. It also allows the user to set the periods of time, within 24 hours, when the system should be in operation.

HomeWarm is made up of the following components.

- ▶ A thermostat, which senses the temperature of the environment and lets the user set the target temperature to be maintained.
- ▶ A boiler/pump, which heats up and circulates the water to the radiators in the house.
- ▶ A control unit, which lets the user set up the periods of operation of the system (start and end times) within 24 hours. The control unit includes a display which indicates to the user the set periods of operation and whether the system is currently in operation.

The *HomeWarm* system is designed as an object-oriented system based on an MVC framework. The system includes the classes shown in Figure 11.

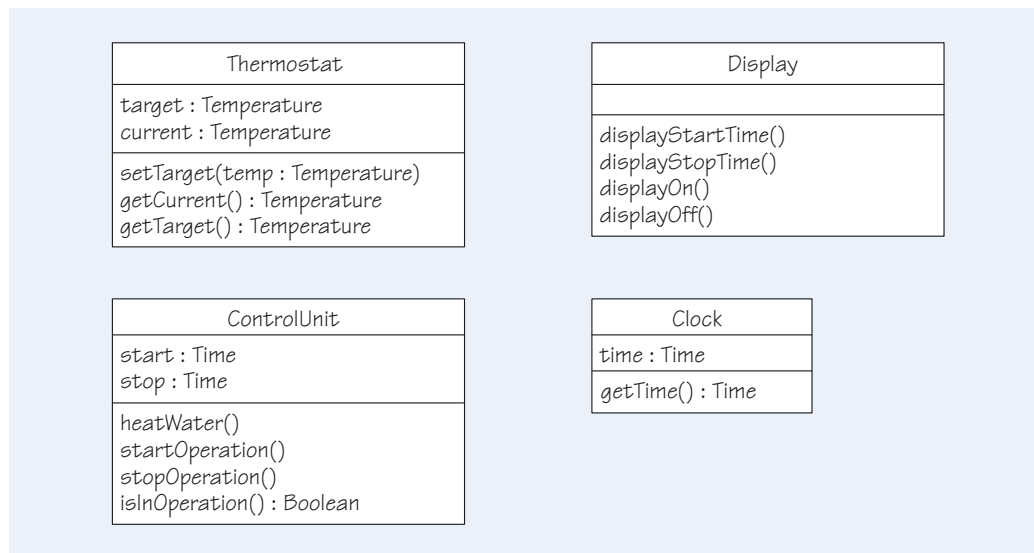


Figure 11 Classes in the HomeWarm system

Thermostat senses the current room temperature and lets the user set the target temperature.

Clock keeps track of time for the control unit.

ControlUnit allows the user to set the system operation period (start and end times) within 24 hours; when it is operating, if the temperature sensed by the thermostat falls below the target temperature, it activates the boiler.

Display shows the current user setting (start and end times), and whether the system is currently in operation.

The Observer pattern is applied repeatedly in the design of *HomeWarm* to achieve the correct system behaviour.

- (a) Identify all the relevant subject/observers dependency relations between the *HomeWarm* classes. Focus on MVC principles and values that change at run-time.
- (b) Draw a sequence diagram to illustrate what happens when the system starts operating and senses a temperature below the target one.

ANSWER.....

- (a) *Display* depends on *ControlUnit* to show whether the control unit is currently in operation and to show the user settings. *ControlUnit* depends on *Thermostat* to find out the current room temperature. *ControlUnit* depends on *Clock* to start and stop operating.
- (b) A possible diagram is shown in Figure 12.

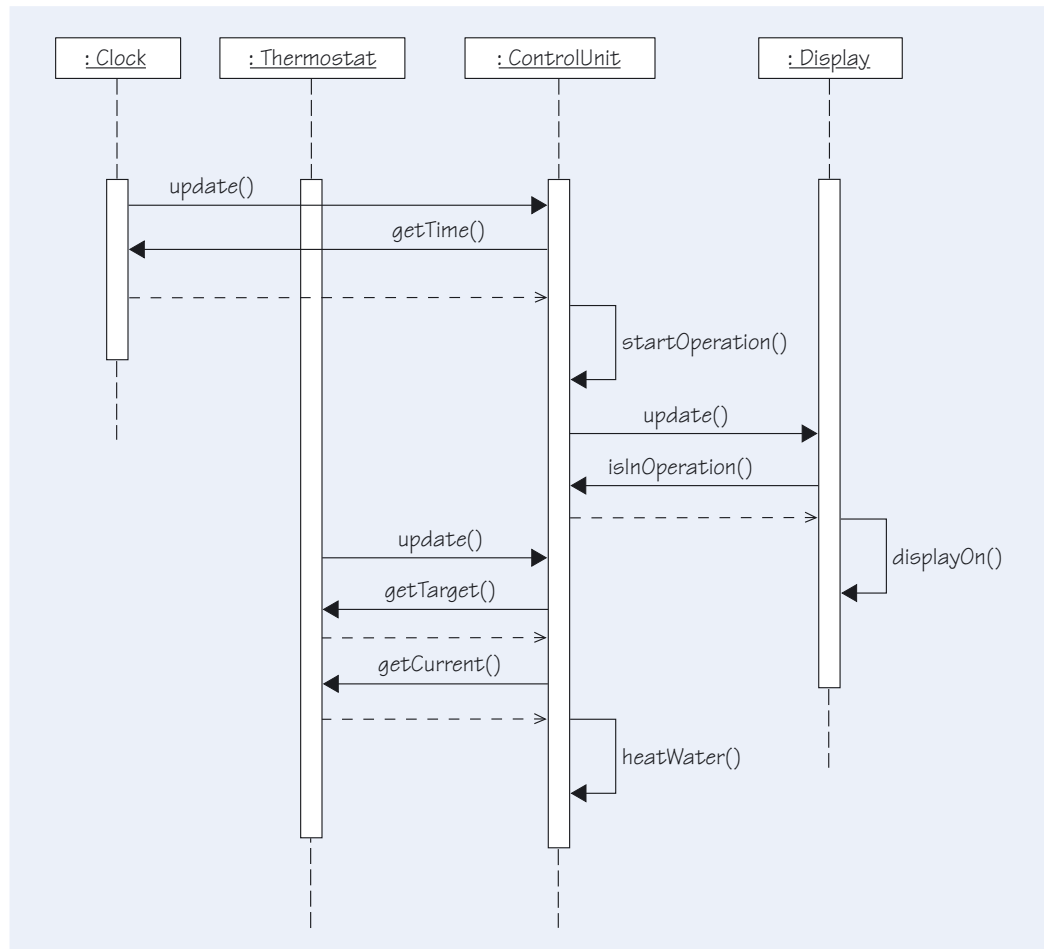


Figure 12 *HomeWarm* sequence diagram

3.2 Singletons and Factories

The two patterns in this section are both well known design patterns, introduced by Gamma et al. (1995). They are used to help solve two specific design problems that recur over and over: how to ensure that only one instance of a class exists, and how to create complex instance hierarchies. The first problem is addressed by the **Singleton pattern**, and the second by the **Factory pattern**.

Before looking at these patterns in detail, we revisit some relevant UML notation.

Interfaces were introduced in Section 4 of *Unit 9*.

In UML, an interface is a description of a set of externally visible operations. It specifies a set of services in terms of operations and their signatures (that is, how they can be invoked). The UML notation for interfaces is given in Figure 13. An interface is not a class. It does not define the operations, only their signature. Instead, operations are defined within classes that *realise* the interface. A class realises an interface by defining (or inheriting) operations corresponding to those of the interface. In Java, interfaces are realised by classes using the `implements` keyword.

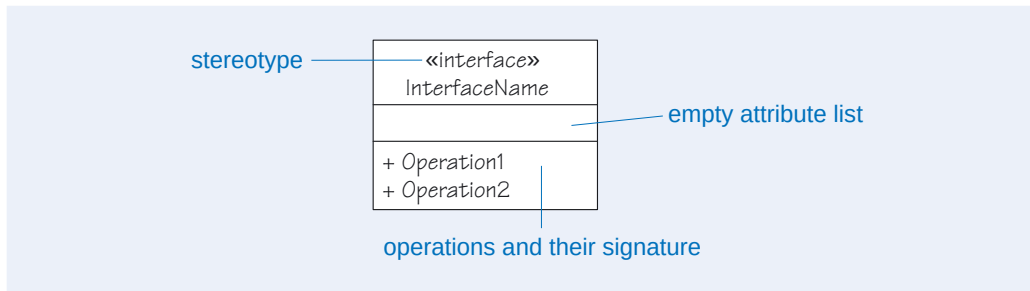


Figure 13 UML notation for interfaces

It is possible for a class to realise more than one interface, in which case the class has to implement all the operations found in all the interfaces. Of course, a class may contain additional operations too. **Realisation** in a class diagram is shown in Figure 14.

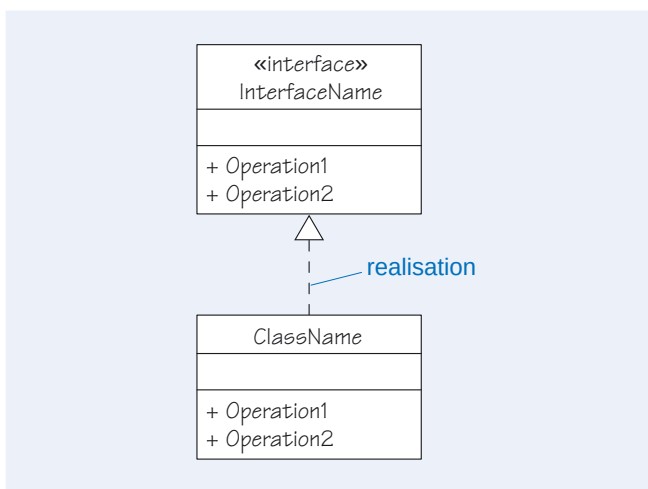


Figure 14 UML notation for realisation

SAQ 10

What is the difference between generalisation (e.g. the superclass in inheritance) and realisation?

ANSWER.....

Generalisation expresses a subtyping relationship between two classes: the subclass is a specialised subtype of its superclass, and instances of the subclass can appear anywhere that instances of the superclass can; the subclass inherits attributes and operations of the superclass. Realisation is not a subtyping relationship, but instead asserts that a class realises a protocol specified by an interface. The realising class provides an implementation for all the operations that are declared in the interface.

Singleton

The Singleton pattern is defined as follows.

- ▶ *Name* Singleton.
- ▶ *Intent* To ensure that a class has only one instance, and to provide a global point of access to it.

- *How it works* The solution is described by the UML class icon in Figure 15. The *Singleton* class defines a public class operation *getInstance()*, that lets other objects (the clients) access its unique class attribute *uniqueInstance*. The *Singleton* class is usually responsible for creating its own unique instance, so the *create()* operation is private. In the figure, the «*singleton*» stereotype highlights the application of the pattern. In Java, if an application is neither distributed nor multi-threaded, *getInstance()* can be implemented as a static method, returning a private static instance.

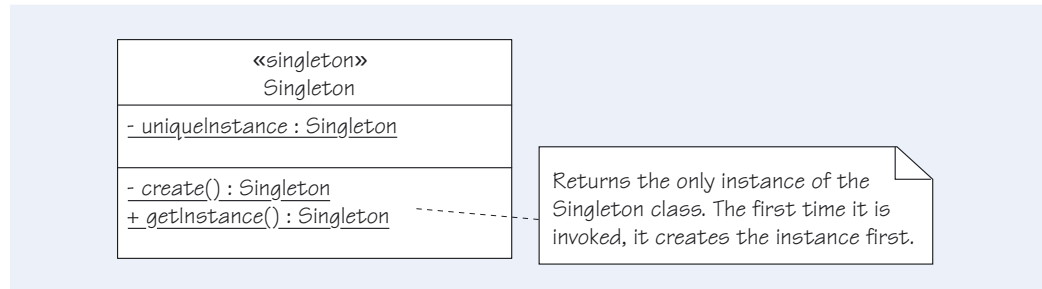


Figure 15 Example UML for the Singleton design pattern

- *When to use it* Use the Singleton pattern when there must be only one instance of a class, which must be accessible from a well known access point.
- *Example* Figure 16 illustrates the application of the Singleton pattern to the design of a media manager within a multimedia application. The idea is that there should be only one instance of the manager, which is created by the class itself the first time a client accesses the manager. In the example, both class attribute *manager* and class operation *create()* are declared as private, while operation *getManager()* is declared as public. In Java, the *manager* attribute and the *getManager()* method could be declared as *static*, permitting them to be used without having an instance of the class. The *getManager()* method would create the one instance and invoke *create()* the first time *getManager()* is invoked, which it knows because the *manager* attribute would still be *null*.

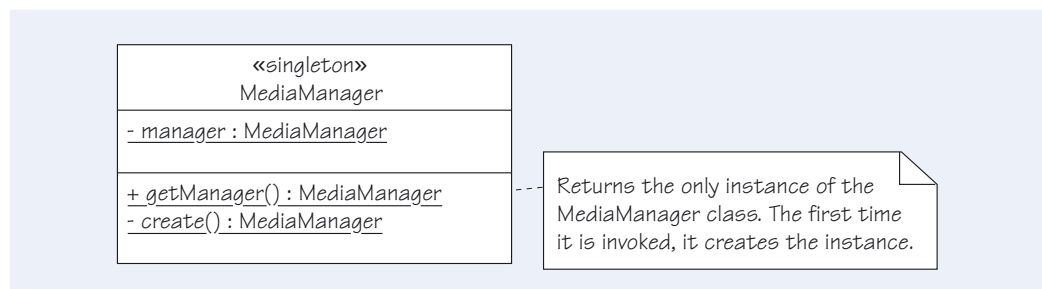


Figure 16 A multimedia manager as an example of the Singleton design pattern

SAQ 11

Discuss a circumstance under which it would be appropriate to use the Singleton pattern in the hotel reservation example shown in Figure 2.

ANSWER.....

If the software was being designed to manage a single hotel chain, then the class *HotelChain* would be required to have only one instance. The Singleton pattern could be applied to enforce this condition. Note that the hotel reservation application is almost certainly both distributed and multi-threaded, so implementing Singleton will be more complex than using static methods.

Factory

The Factory pattern is defined as follows.

- ▶ *Name* Factory.
- ▶ *Intent* To define a specialised object for the creation and initialisation of other objects.
- ▶ *How it works* The specialised object is called a *factory*, while the object (or objects) to be created is called the *product*. The pattern works by defining a factory for the creation of products. Clients requiring products for some purpose use the factory to create them, rather than creating them directly or indirectly by assigning responsibility to some other functional class, as was done in Figure 14 of Unit 8 (reproduced in this unit as Figure 21). In Figure 21, responsibility for creating a reservation was assigned to *Hotel*. The structure of the pattern is shown in Figure 17, where the client depends (dashed line) on the factory for the creation of the product, and the factory depends on the *Product* class to create one of its instances. The UML stereotype «create» indicates that the dependency between the two classes occurs at object creation. The dependency expresses the fact that within *Factory* the name of class *Product* has to be known in order to create an instance of that class. Also, as the factory has to be easily accessible, it is realised as the only instance of a Singleton class.

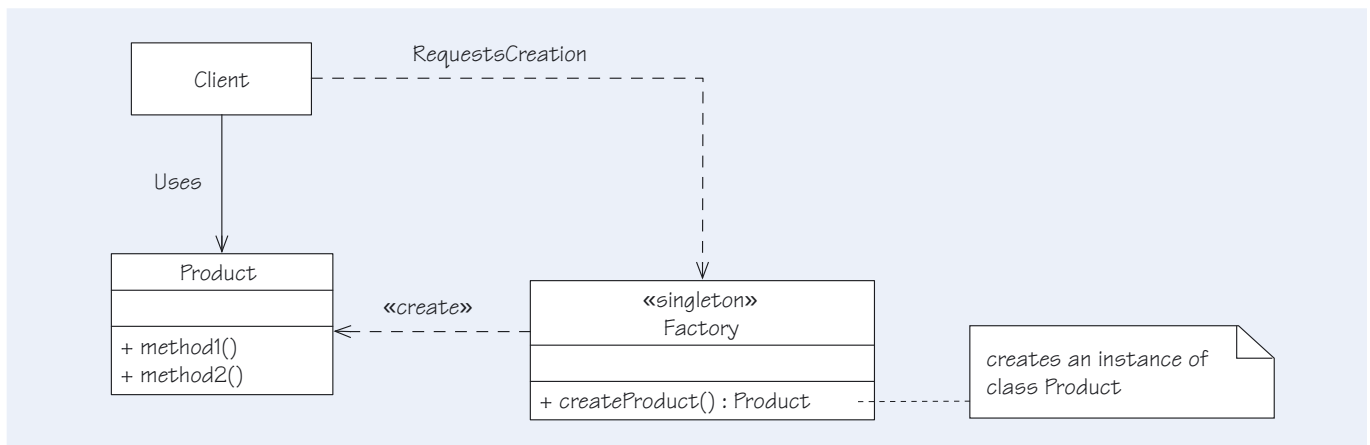


Figure 17 UML structure for the Factory pattern

A variant of this pattern (known as the **Factory method**) is shown in Figure 18. In this case, both factory and product are abstracted through the use of interfaces. The client relies on only the given interfaces for the creation and use of the product. This version of the pattern is particularly useful in the definition of frameworks. In a framework, application-specific factories and products can be defined based on the interfaces defined in the framework and the problems the framework is intended to help solve.

You can find a Java example of the Factory pattern on the course website. Note that in this example, using Singleton was unnecessary because there is only one instance of each factory.

An implementation of the same application, but using Singleton, can be found on the course website. This example assumes no multi-threading and that the application is not distributed.

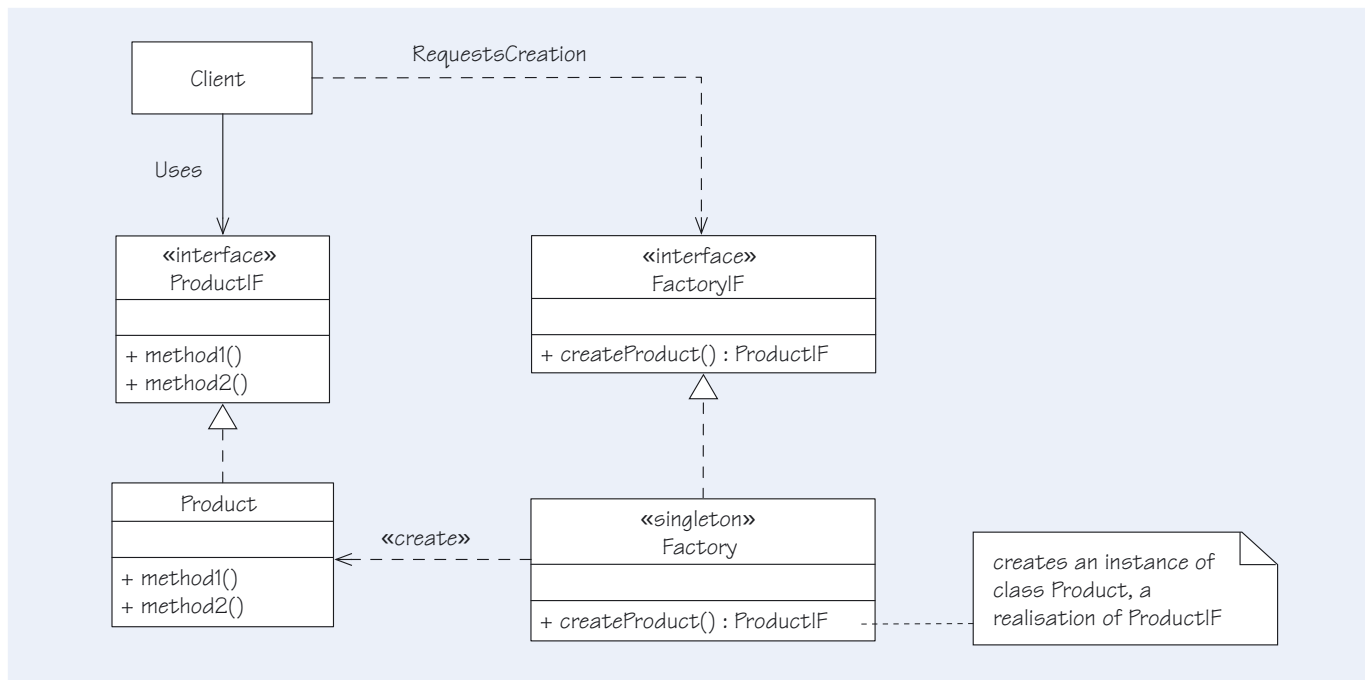


Figure 18 *Factory method* variant of the Factory pattern

- ▶ **When to use it** This is a general pattern for object creation and initialisation which increases the flexibility of the system. By localising the creation and initialisation of objects within the factory, clients are isolated from changes in the implementation of the product, such as creation order considerations. By applying the Factory method variant, clients can be completely isolated from the realisation of the factory and its products (i.e. the classes realising the interfaces).
- ▶ **Example** Assume there is a framework for document generation, in which applications can define application-specific documents which can be manipulated by an editor. The application of the Factory method is shown in Figure 19.

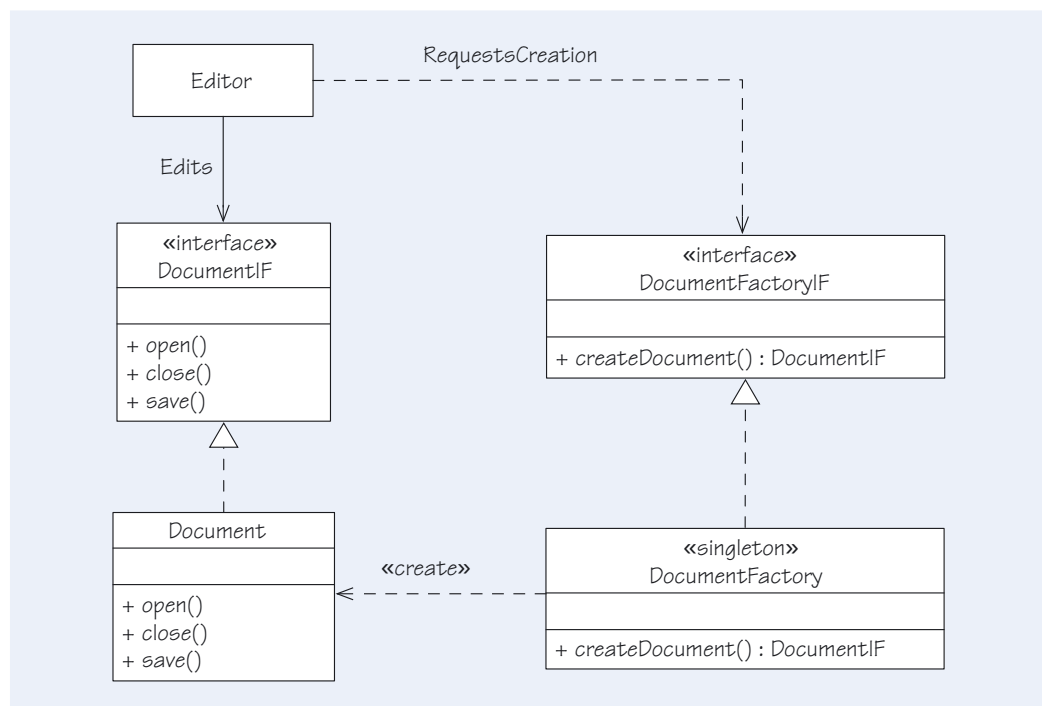


Figure 19 Example using the *Factory method* variant of the Factory pattern for document generation

SAQ 12

Given the document factory example above, how would you modify the class diagram to include a factory for a new type of document? (Hint: None of the interfaces should be modified.)

ANSWER.....

A possible solution is given in Figure 20. A new class representing the factory is included as well as a class representing the new document. Creation requests to the new factory (*AnotherDocumentFactory*) will result in creation of instances of *AnotherDocument*.

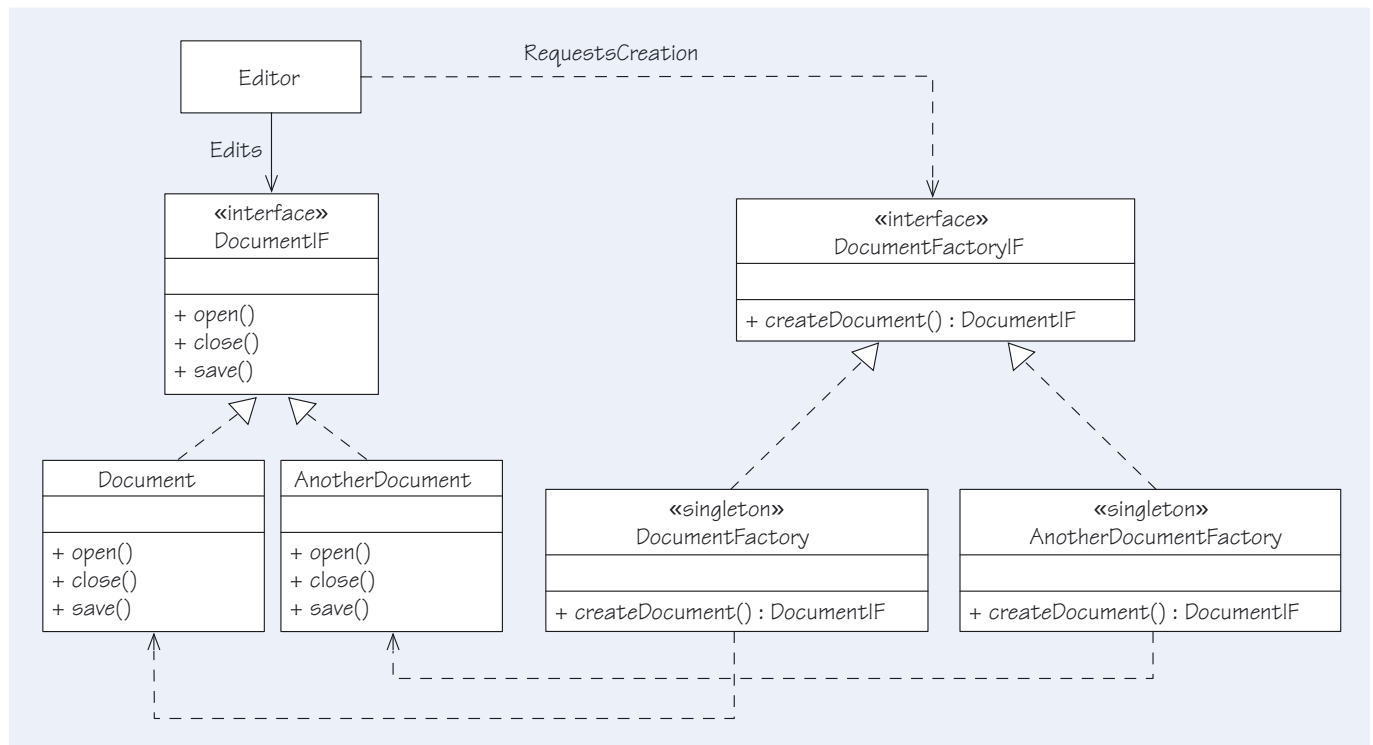


Figure 20 The document factory with a new document type

Factories and GRASP Creator

Recall that GRASP Creator is used to assign responsibilities related to the creation of objects. Assigning these responsibilities can get complicated when faced with a hierarchy of objects with links between objects inside the hierarchy. Factories are one way of avoiding or reducing some of these complications.

The GRASP Creator pattern was introduced in *Unit 6*.

Consider the discussion in *Unit 8* that led to Figures 14 and 15 in that unit, reproduced here as Figure 21 and Figure 22 respectively. The discussion revolved around two choices for assigning responsibility for creating a new *Reservation*. In Figure 21 the responsibility is assigned to *Hotel*, and Figure 22 shows what happens if the responsibility is assigned to *ReserverPayer*. In the first case, *Hotel* must be aware of the link between *ReserverPayer* and *Reservation*. In the second case, *HotelChain* must be aware of the link. This awareness is visible in Figure 21, where *Hotel* passes an instance of *ReserverPayer* to *Reservation*, and in Figure 22, where *HotelChain* delegates the creation to *ReserverPayer*. Both choices increase coupling within the system because *Hotel* or *HotelChain* must know details about the links between the objects that they are linked to. A factory eliminates this requirement, because the details are encapsulated in the factory. In addition, maintenance problems are reduced because the details of the implementation, such as order of instance creation and implementation of the associations, are also encapsulated in the factory and not scattered throughout the system.

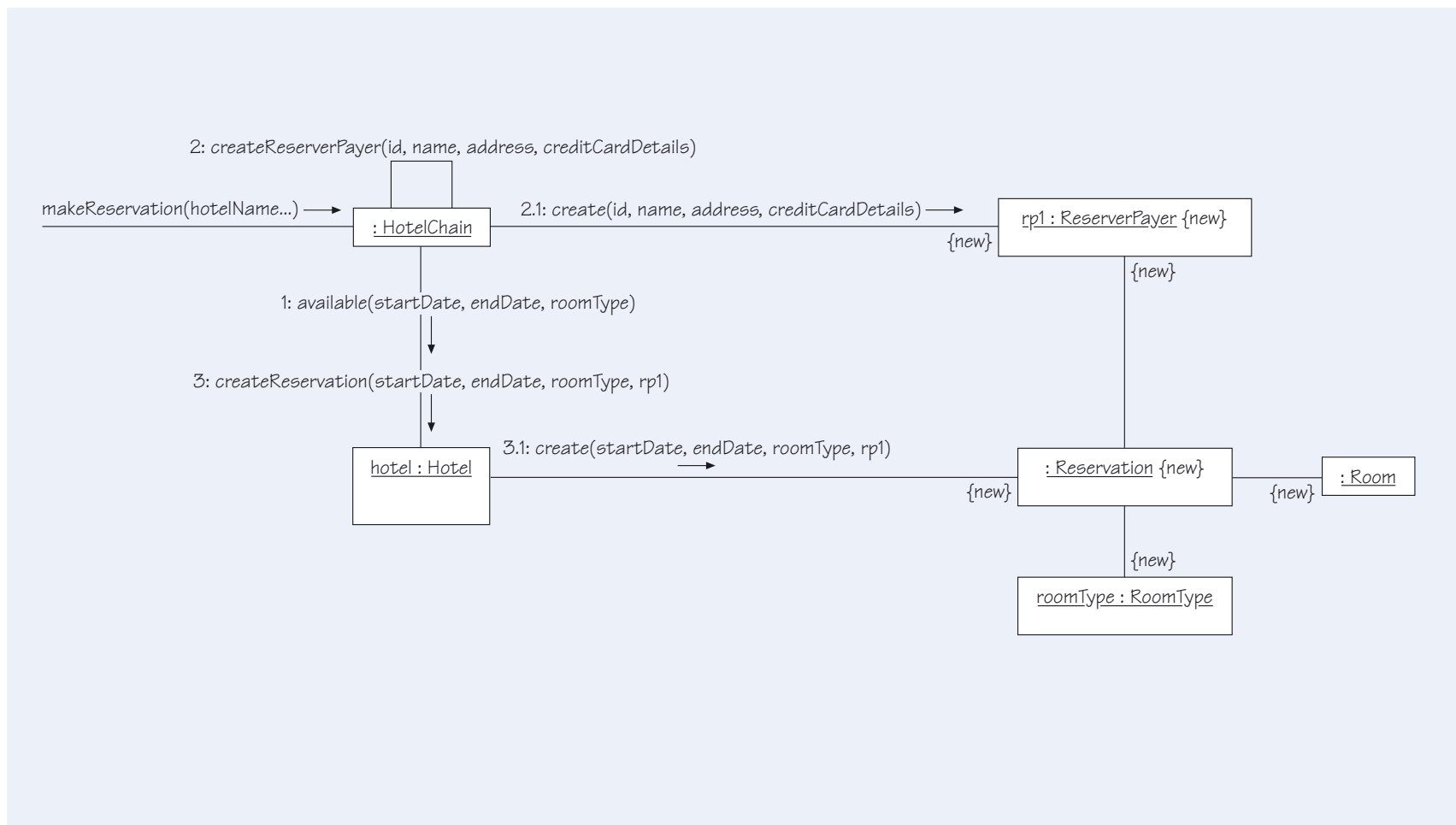


Figure 21 Communication diagram for the fulfilment of the postcondition of *makeReservation* (reproduction of Figure 14 from *Unit 8*)

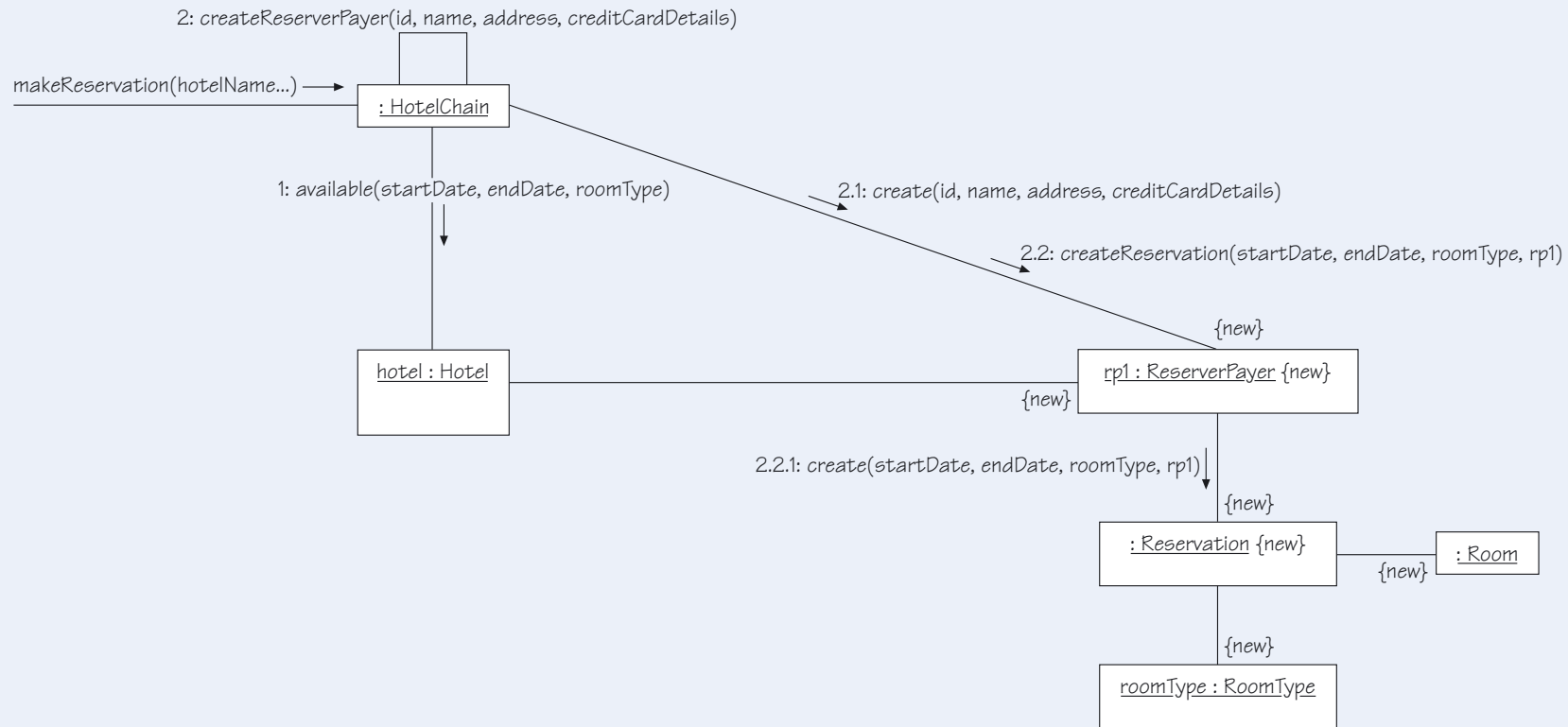


Figure 22 Revised communication diagram for the fulfilment of the postcondition of *makeReservation* (reproduction of Figure 15 from Unit 8)



Exercise 3

Consider the collaboration for the postcondition ‘the system has a new item whose attribute is initialised to the given code’ for the system operation *addAnItem()* shown in Figure 23.

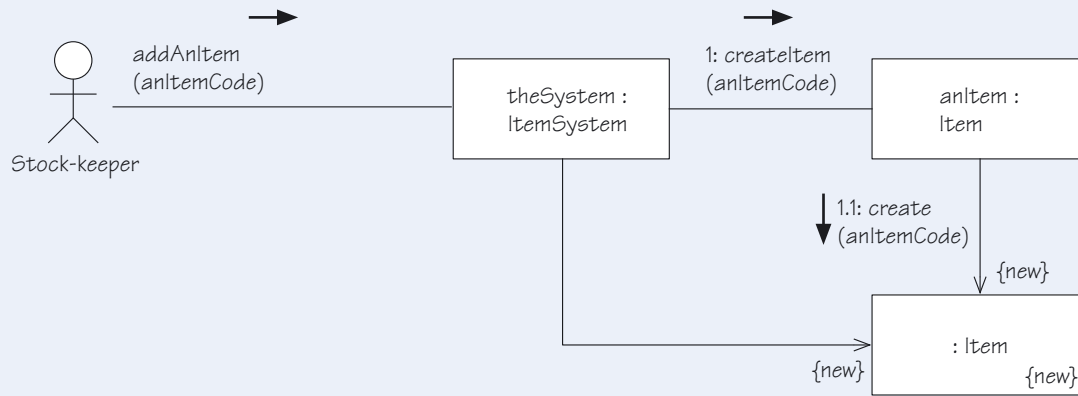


Figure 23 Communication diagram for adding a new item

Based on such a design:

- define a factory class for the creation of new instances of *Item* and the establishment of appropriate links.
- give the operation contract of the operation in the factory class which realises the creation and initialisation of an *Item* object.
- illustrate the new collaboration involving the factory in a sequence diagram.

Solution

- The concrete Factory class icon is shown in Figure 24.

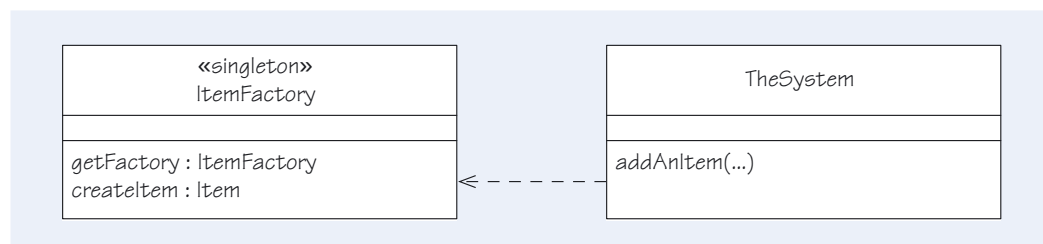


Figure 24 Factory class for adding a new item

(b) The new collaboration is illustrated in the sequence diagram shown in Figure 25.

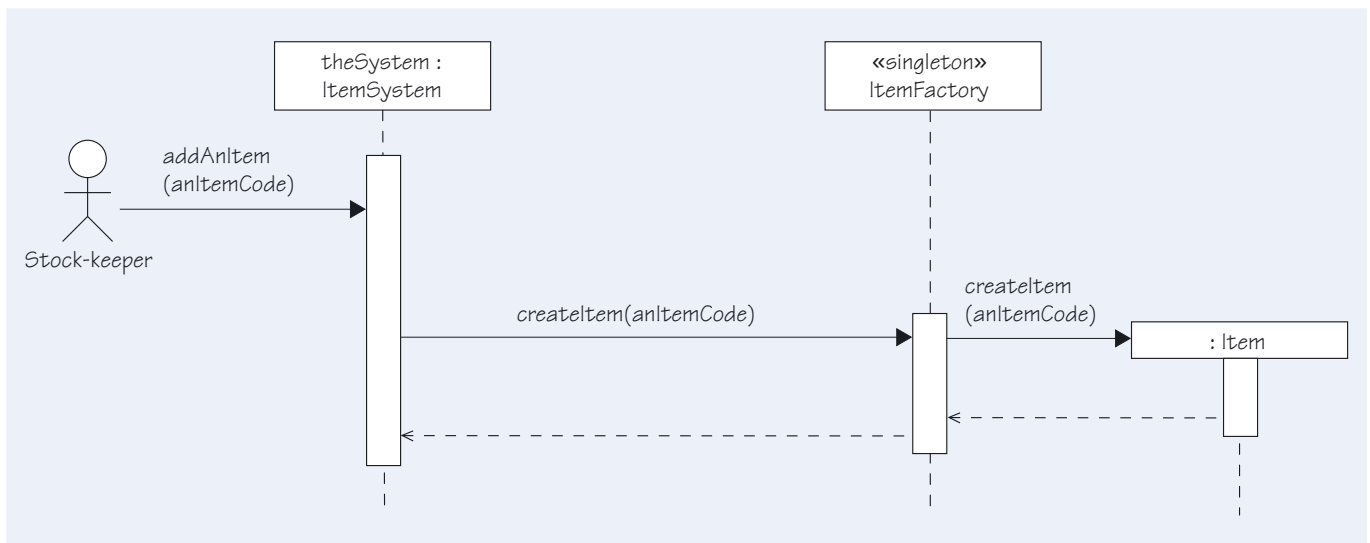


Figure 25 Sequence diagram for adding a new item

Exercise 4

Discuss some advantages of using factories for the creation of objects rather than applying GRASP Creator directly to classes of the domain layer.



Solution

One advantage is that the logic for the creation and initialisation of objects of a given class is encapsulated within the factory. This improves system flexibility as changes to how the objects are created and initialised will be local to the factory. Also, the coupling within classes in the system is reduced for the following reasons:

- ▶ The factory knows how to create and initialise a hierarchy of objects such as *ReserverPayer* and *Reservation* in Figure 22, removing that knowledge from the *ReserverPayer* and *HotelChain* classes.
- ▶ The delegated creation methods such as *createReservation* in *ReserverPayer* (again in Figure 22) are not necessary. The factory knows that there must be a link between a *ReserverPayer* and its reservations, so that *HotelChain* need not be concerned with this detail.
- ▶ By implementing a factory as a singleton, the factory can be accessed without the need for links between the factory object and its clients.

3.3 Summary of section

In this section you have seen some patterns for application layering: the Model-View-Controller pattern, the Application Controller pattern and the Observer pattern. You have also seen the Singleton and Factory patterns.

4 Enterprise components and architectures

Enterprise-scale component-based development requires more than a disciplined application of the basic component design principles. Adherence to standards is essential, because it allows different organisations and manufacturers to produce and market components that can be acquired and plugged together to build new systems. This is common in other engineering disciplines, for instance electrical engineering. Here, commercially available standard components which have specific, clearly defined functions can be plugged together following some well defined rules, irrespective of their manufacturer or brand name. For instance, in the UK, any 3 amp fuse can be inserted into just about any appliance plug, and it will function properly as long as the amperage is appropriate for that appliance.

Industry standards have emerged for software components too, although they are not nearly as well established and stable as electrical standards. In this section we look briefly at one such standard.

Java Bean definition from the Sun Developer Network Tutorial and Code Camp article *Java Beans, Pt. 1: Definition: What is a Bean?* found at <http://java.sun.com/developer/onlineTraining/Beans/Beans1/simple-definition.html> (Accessed December 2007)

Enterprise JavaBeans (EJB) is a component technology from Sun Microsystems, based on the Java programming language. According to Sun Microsystems, 'A Java Bean is a reusable software component that can be visually manipulated in builder tools.' EJB extends this definition by defining a standard that helps organisations to build their own components or acquire components from third-party vendors. EJB components are also called **enterprise beans**. They are server-side components, that is, they are distributed entities that provide remote services to (software) clients distributed throughout the network.

To function, enterprise beans have to be deployed in a run-time environment called an **Enterprise JavaBeans container**. This is shown in Figure 26. The container manages remote access to the bean, together with providing a number of other functions, including security, persistence, transaction and resource management. The container isolates the bean from its clients so that they do not have direct access to it; the bean is totally reliant on the container with which it communicates via a standard protocol defined in the EJB specification.

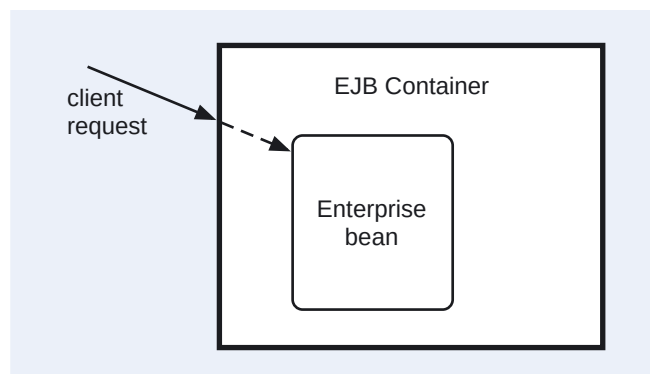


Figure 26 EJB container

One can access an enterprise bean locally and remotely. A local access is one where the enterprise bean container and the accessing client must be on the same computer. A remote access is one where the container and client may be (but are not required to be) on separate computers. This distinction is relevant to performance; accessing a remote bean requires more time, even if the access turns out to be to the same computer.

Creating enterprise beans has been made much simpler with the EJB version 3 specification. Previously, the developer had to write complex interfaces and XML description files. The version 3 specification helps the developer concentrate on what the bean should do, leaving the complexities to the compiler and run-time environment. In version 3, to create an enterprise bean, the developer first specifies the business functions that the bean is to make available by writing a normal Java `interface`, which in the version 3 specification is called a *Plain Old Java Interface* (POJI). Next, the developer implements the desired functionality as a *Plain Old Java Object* (or POJO); this class `implements` the business function interface. The POJO class can implement more than one business interface. The developer then uses *metadata annotations* on both the POJI and the POJO to tell the Java system that the object is an enterprise bean, what kind of bean it is, and some characteristics of the bean. A metadata annotation is a keyword beginning with the `@` character, possibly followed by parameters. For example, the `@Remote` annotation on the POJI indicates that the bean must support remote access through that interface, while the `@Local` annotation on the POJI indicates that the enterprise bean is to support only local access through that interface.

There are two basic types of beans in EJB:

- ▶ **session beans:** these represent (sets of) business functions;
- ▶ **message-driven beans:** these support asynchronous processing through message passing.

We illustrate an enterprise session bean with a small example, a business service to convert pounds to euros. This is an example of a session bean that provides a service but does not have any state (does not remember anything from invocation to invocation). Figure 27 shows the POJI, Figure 28 the POJO, and Figure 29 shows the client that uses the bean.

```
package toyEJB;
import javax.ejb.Remote;
import java.math.*;

@Remote public interface Converter {
    BigDecimal poundToEuro(BigDecimal pounds);
}
```

Figure 27 The interface (POJI) for the example enterprise bean

The interface in Figure 27 is annotated with the `@Remote` annotation, indicating that the business service can be used in a distributed system (over a network).

The following Java implements the business service. The `@Stateless` annotation tells the compiler that the class is a session bean, and that nothing is remembered between invocations. The class implements the POJI `Converter`, making the connection between the business service implementation and the interface that the clients see and use.

Formerly there were also entity beans, but these are now treated as classes.

The example presented in this unit is based on the example in the Sun Developer Network article *Ease of Development in Enterprise JavaBeans Technology*, found at <http://java.sun.com/developer/technicalArticles/ebeans/ejbease> (Accessed December 2007).

```

package toyEJB;
import javax.ejb.Stateless;
import java.math.*;
import toyEJB.*;

@Stateless public class ConverterBean implements Converter {
    final BigDecimal euroRate = new BigDecimal(1.48);

    public BigDecimal poundToEuro(BigDecimal pounds) {
        return pounds.multiply(euroRate);
    }
}

```

Figure 28 The business services class (POJO) for the example enterprise bean

The client of this business service is required to inform the Java compiler that it is using an enterprise bean. This is done using the `@EJB` annotation. The created object that represents the enterprise bean (`converter`) is required by the EJB specification to be static.

```

package ToyEJB;
import javax.ejb.EJB;
import java.math.*;
import toyEJB.Converter;

public class Main {
    @EJB private static Converter converter;
    public Main() {
    }
    public static void main(String[] args) {
        Main me = new Main();
        me.doTest();
    }
    private void doTest() {
        System.out.println("----- IN TEST -----");
        BigDecimal pounds = new BigDecimal(101.00);
        System.out.println(pounds + " pounds equals "
                           + converter.poundToEuro(pounds)
                           + " euros");
        System.out.println("----- DONE -----");
    }
}

```

Figure 29 A client of the example currency conversion enterprise bean

Note that the above is a simple example. One can do much more with annotations, such as indicating that a method be called when the bean is created or destroyed.

SAQ 13

Summarise the steps you need to take to define an enterprise bean.

ANSWER.....

To define an enterprise bean it is sufficient to define one interface and one class. The interfaces represent the business functions supplied by the bean, which clients need to make use of the bean. The class represents the implementation of the bean and provides the code for all the methods declared in its interfaces.

4.1 Component architectures

EJB is a component technology aiming to support the development of server-side business components. Other standards have emerged which aim to support many other aspects of enterprise application development, from user interaction to database integration. The ultimate goal is for whole system architectures to be built by fitting standard components together. Solutions that provide this sort of support are known as **component architectures** or **common infrastructures**. In this section we look briefly at one such architecture.

The Java platform, Enterprise Edition (Java EE), is a component architecture for developing distributed enterprise software applications. Java EE provides a range of technologies for the development of applications through the creation, integration and deployment of components. Such technologies provide a wide range of services, which are made available to developers through standard application programmable (or programming) interfaces (APIs). An API defines a set of operations that can be invoked on (sets of) components to make use of their services while developing a software application, without knowing any implementation details. Through such services, the API platform provides an infrastructure for enterprise applications which deal with complex issues, such as resource allocation, transactions or security, and allow the developer to concentrate on supporting business processes.

Like EJB, Java EE is based on the Java programming language and is a trademark of Sun Microsystems. In fact, EJB is part of Java EE. Java EE is an aggregation of many technologies that have been standardised over the years and integrated into the Java EE platform, which continues to evolve and grow as new technology standards emerge.

Java Community Process (JCP)

The Java Community Process (JCP) was introduced by Sun Microsystems in 1998 as an open, participative process to develop and revise the Java™ technology specifications, reference implementations, and test suites. Since then, the JCP programme has fostered the evolution of the Java platform in cooperation with the international Java developer community.

Individuals or organisations can become JCP members by signing the Java Specification Agreement (JSPA). The JSPA is an agreement between a company, an organisation or an individual and Sun, setting out each community member's rights and obligations when participating in the development of Java technology specifications in the JCP.

While there are no mandatory obligations, the success of JCP is predicated on members' participation in a range of activities, including providing feedback on proposed specifications, submitting specifications, leading or nominating people to form expert groups or build independent implementations.

The JCP website address is <http://jcp.org/en/home/index> (Accessed December 2007).

A driving goal behind the development of Java EE was supporting application distribution and portability. Enterprise applications developed in Java EE are amenable to being distributed across computer networks and, by and large, they work on all operating systems. This is possible because of the separation between Java EE applications, their infrastructure and the operating system: many

vendors provide Java EE infrastructures for various operating systems and a Java EE application can be deployed in any of these infrastructures.

Among the many technologies that make up the Java EE platform, the following are of particular significance.

- ▶ *J2SE (Java 2 platform Standard Edition)*: This is the foundation for all technologies in Java EE. It contains all the Java programming language interfaces and libraries. Developers use J2SE as the primary API to develop Java applications.
- ▶ *EJB (Enterprise JavaBeans)*: As you saw in the previous section, this technology provides a standard for developing reusable server-side business components, called enterprise beans. Application-specific enterprise beans can be developed using the standard API provided by the platform. Among the services provided by the platform for EJB components are portability, persistence, distribution and transaction processing.
- ▶ *JSP (Java Server Pages) and servlets*: These are technologies for building the server parts of web applications.
- ▶ *JMS (Java Message Service)*: This technology allows for standardisation of messaging, a way for software components to communicate asynchronously.
- ▶ *JDBC (Java DataBase Connectivity) and Java Persistence API*: These technologies allow for standard database access from applications developed in Java.

J2ME stands for Java 2 Micro Edition. It is an optimised platform for wearable and mobile devices.

Support for component-based application within Java EE relies on the notion of a **container**. In Java EE, a container is a generic infrastructure for components, providing a range of services to support such components; and many specialised containers provide specialised infrastructures. You have already encountered the EJB container for enterprise beans (discussed above); other containers are illustrated in Figure 30, which gives a summary of the Java EE platform.

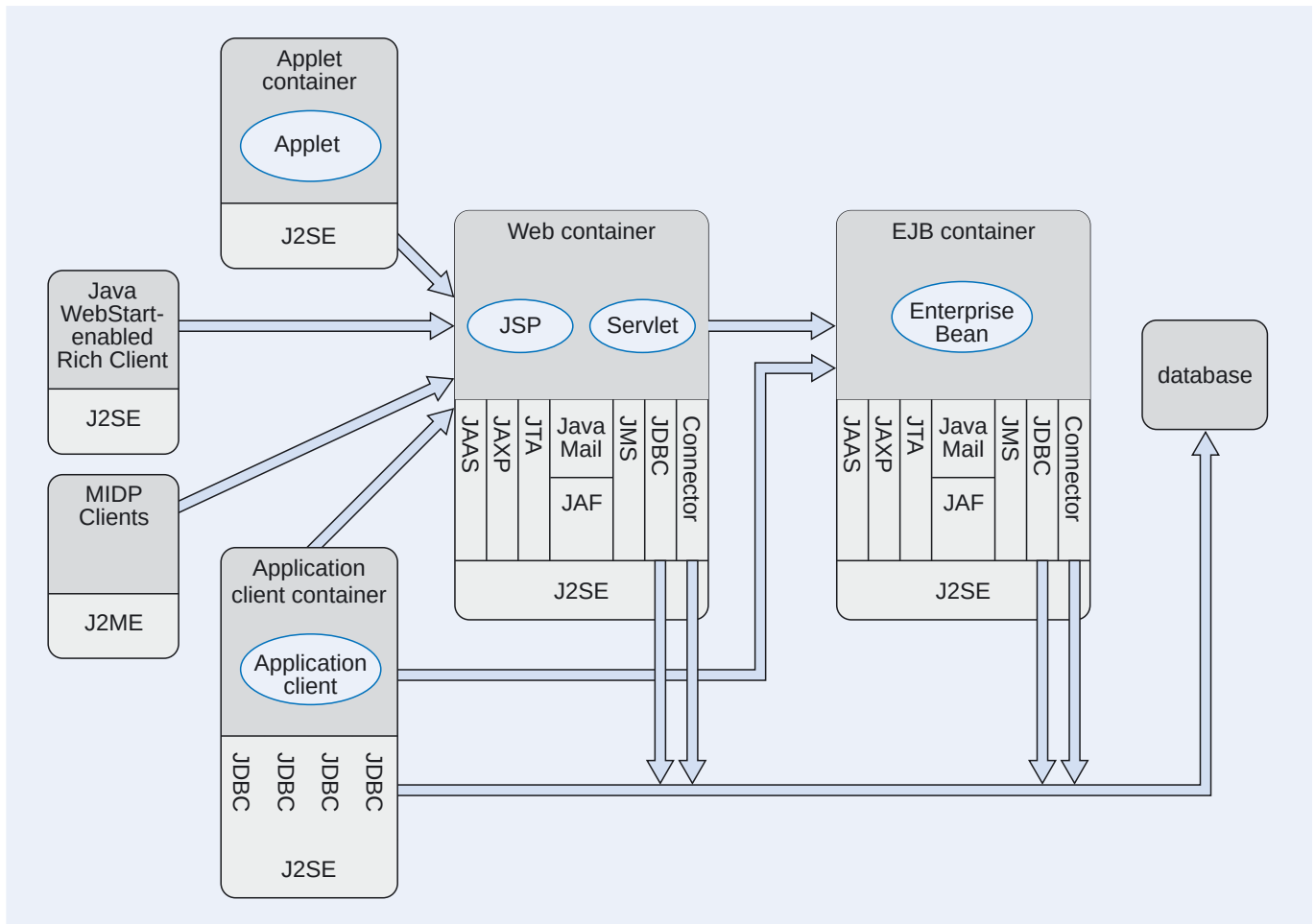


Figure 30 The Java EE platform (copyright Sun Microsystems)

It is not important that you understand each bit of technology in Figure 30 – it is just to give you an idea of the wide range of technologies that are available under the Java EE architecture.

Each Java EE container includes some key technologies which provide the infrastructure for components (such as J2SE, JDBC, JMS), as well as specific component technology for that container. The Java EE containers are as follows.

- ▶ *Client containers:* These are containers for client components. Java EE supports different types of clients, from web pages to dedicated client applications and mobile devices. Component technologies here include Java applets (limited Java applications that run within a web browser), and a collection of application client technologies.
- ▶ *Web container:* This includes component technology to support web clients. Relevant component technologies here are JSP and servlets.
- ▶ *EJB container:* This is for EJB components, as explained above.

For more specific information on Java EE, you can visit the Java EE home page, at <http://java.sun.com/javaee> (Accessed December 2007).

SAQ 14

Java EE applications follow the Layers architectural pattern. Discuss where Java EE containers play a role in a layered architecture.

ANSWER.....

With reference to three-layer architecture, client containers and applet containers work in the presentation layer, while the EJB container works in the domain layer. Technologies such as JDBC, which are available in most containers, provide an interface to databases and hence integration with the data source layer.

One of the strengths of Java EE is the fact that it allows developers to pick and choose flexibly from a wide range of technologies in order to develop their applications. The possible resulting architectures are many, from a system made up of a single application client (so-called fat client) which 'talks' directly to a database through JDBC, to a complex distributed application which uses just about every bit of available technology.

The term 'client' is often applied to those parts of the application that are used by actors in a use case diagram.

SAQ 15

Suggest how you could use Java EE technology to build the client parts of the hotel reservation system.

ANSWER.....

The hotel reservation system will require two types of clients: a web client to allow customers to make reservations online, and a client application to be installed in the hotels. Therefore, application client and web container technologies will be required. The business logic may be implemented using enterprise beans within an EJB container, from which the system will be able to integrate with legacy databases. A possible architectural outline is shown in Figure 31. Note that in practice, in addition to knowledge of presentation, servlets can contain a significant amount of domain knowledge such as how database tables are interconnected. Because of this, some people put servlets into the domain layer.

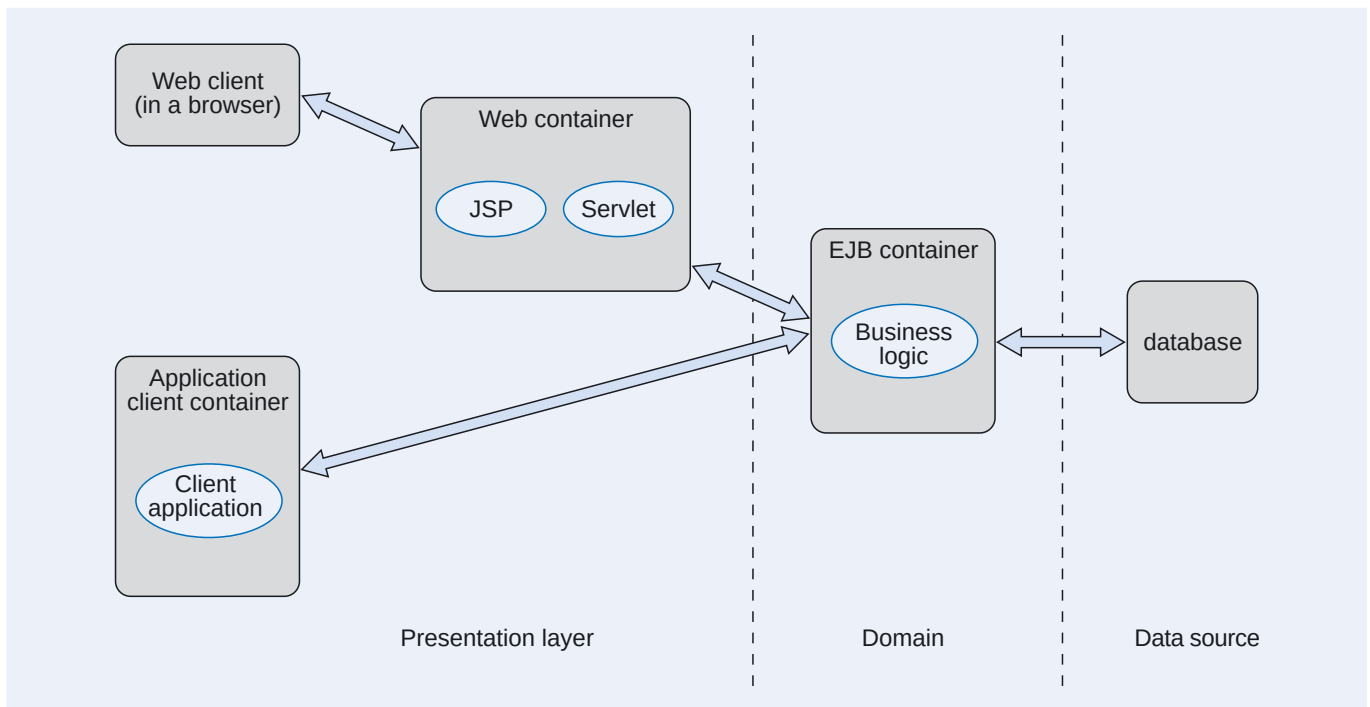


Figure 31 Possible Java EE architecture for the hotel reservation system

4.2 Service-oriented architectures

This section introduces one of the latest trends in enterprise application development, that of **service-oriented architecture (SOA)**. At the time of writing, this is more an emerging trend (albeit an important one) than established practice. However, both industry and academia are investing heavily in this approach and a range of standards and supporting technologies have been produced. It is forecast that SOA will soon dominate enterprise development as the next wave of architecture design.

With information systems increasingly at the heart of business organisations, the main goal of SOA is to provide a technology which supports a flexible business model that can respond quickly to changes in customers' requirements, new business opportunities or competitive threats. Business integration and flexibility as goals are of course not new, but they have become imperatives in today's organisations, due to the advent of the internet, globalisation and e-business. These have accelerated the pace of change and there is now a focus on the extended supply chain, with the need to enable customers and partners to gain access to business services.

SOA is an approach to build enterprise systems that deliver application functionality either as services to end-user applications or to build other services. In this approach, services are software assets, which are described and interact in a standard way, so that the focus of application development is the assembly of services as building blocks.

A **service** is an application function which is packaged as a reusable component for use in a business process. The way a service is actually implemented is not relevant as long as the service exhibits the following key qualities:

- ▶ **interoperability**, which allows clients to communicate with the service: this is achieved by compliance with standard communication protocols;
- ▶ **location transparency**, which allows clients to make use of the service without being aware of where the service is actually located, i.e. on which machine or even in which business organisation. The client of the service and the service

itself are only loosely bound; in particular, the client has no knowledge of the service implementation or its platform, and they only communicate through messages.

There are similarities between a service and a component. They both promote reuse and flexibility. They both use public interfaces to allow requesters to make use of their functionality, without relying on their implementation. There are differences too. A component is usually realised in a specific technology, often object-oriented, therefore only clients compliant with that technology can easily communicate and integrate with it. A service uses communication standards independent of implementation technologies for components, thus allowing more diverse parties to interoperate. While an object-oriented component follows strong encapsulation principles of data and processes, a service can be more flexible in representing a function in isolation (say, applying a tax rate), some data (say, a product catalogue), or a combination of both. Finally, components tend to be associated with business entities, while services tend to be associated with business processes: they may provide part or the whole of the functions within such a process and may involve several business entities.

SAQ 16

Draw a table to show the similarities and differences for a service and a component.

ANSWER.....

Table 2 Similarities and differences for a service and a component

	Service	Component
Reuse and flexibility	Yes	Yes
Public interfaces allowing users to use their functionality without understanding their implementation	Yes	Yes
Communication	Uses communication standards independent of implementation technologies	If it is object-oriented, it can only communicate and integrate with object-oriented clients
Encapsulation	Functions and data can be separated in different services	Follows strict rules of encapsulation
Used for	Representing business processes	Representing business entities

One important characteristic of services within SOA is that they are **discoverable**, that is to say the consumer does not need to know about the service or where is located. The SOA infrastructure is able to identify the appropriate service on behalf of the consumer. The way this works is illustrated in Figure 32.

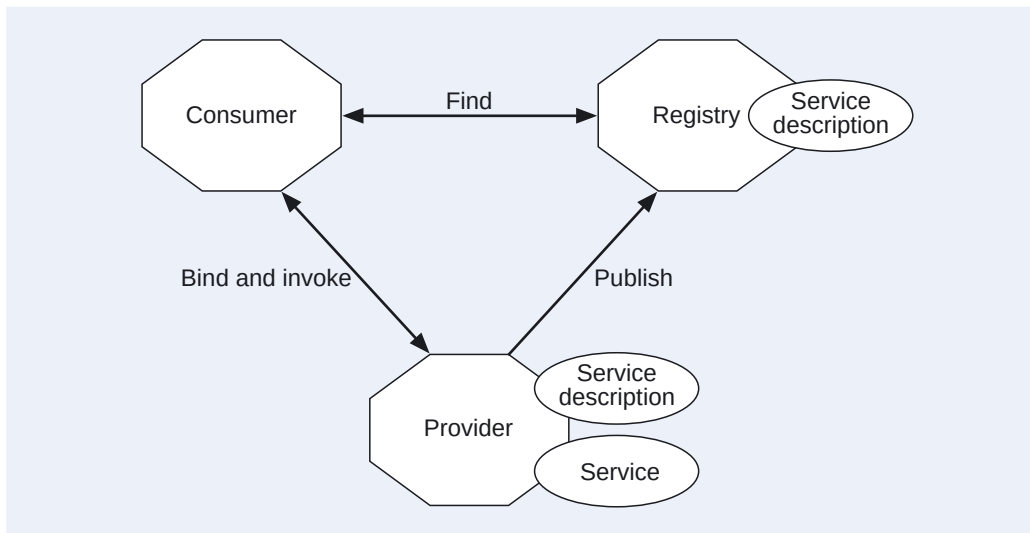


Figure 32 Service collaboration in SOA: the ‘find, bind and invoke’ cycle

The elements in Figure 32 are as follows.

- ▶ *Service*: This is a logical entity; it comes with a description which specifies the way the consumer is expected to interact with the provider. This is essentially a contract: it defines all interfaces and the pre- and postconditions.
- ▶ *Provider*: This is the software entity that implements the service; it accepts and executes requests from consumers.
- ▶ *Consumer (or requester or client)*: This is the software entity which calls a service provider to request a service.
- ▶ *Registry (or locator)*: This is a software entity, which allows the lookup of services, service providers and their location – in other words it allows the service to be found.

The collaboration between parties in an SOA follows the ‘find, bind and invoke’ cycle illustrated in Figure 32: the consumer queries the registry for a service that matches its criteria; if such a service exists, the registry provides the consumer with the contract and the address of the service provider; the consumer can then bind dynamically with the service provider and invoke the service. As shown in Figure 32, for a service to be discoverable, its service description has to be published in the registry.

SAQ 17

Give a summary of the characteristics of a service in SOA.

ANSWER.....

A service is a self-contained and modular software entity. It interacts with applications and other services through a loosely coupled, message-based communication model. It supports interoperability through reliance on communication standards. It can be found by its consumers through a registry; hence its location is transparent to them.

4.3 Summary of section

In this section you have looked at component architectures, including Java EE and EJB. You also learnt a little about service-oriented architecture as an emerging architectural standard for enterprise applications.

5

Summary

In this chapter you were introduced to architectural and design patterns. You learnt about components and how to represent them in UML. You have also looked briefly at component architectures and standards, including Java EE and EJB. You learnt about service-oriented architectures as an emerging architectural standard for enterprise applications, and their current implementation in web services.

LEARNING OUTCOMES

On completion of this unit you should be able to:

- ▶ discuss what is meant by software architecture, as well as views in architectural design;
- ▶ describe and apply the Layers architectural pattern;
- ▶ discuss the benefits of applying design patterns, and describe and apply the MVC, Application Controller, Observer, Singleton and Factory patterns;
- ▶ describe what frameworks and product lines are and what they might be used for;
- ▶ discuss the characteristics of software components and model simple components in UML;
- ▶ give a definition of component architecture, describe the main elements of Java EE and specify simple components in EJB;
- ▶ give a definition of service-oriented architecture (SOA) and discuss its main advantages.

References

- Shaw, M. and Garlan, D. (1996) *Software Architecture: Perspectives on an Emerging Discipline*, Addison-Wesley.
- Bass, L., Clements, P. and Kazman, R. (2003) *Software Architecture in Practice* (2nd edn), Addison-Wesley.
- Kruchten, P. (1999) *The Rational Unified Process*, Addison-Wesley.
- Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P. and Stal, M. (1996) *Pattern-Oriented Software Architecture Volume 1: A System of Patterns*, Wiley.
- Jackson, M.A. (2001) *Problem Frames: Analysing and Structuring Software Development Problems*, Addison-Wesley.
- Fowler, M. (1997) *Analysis Patterns*, Addison-Wesley.
- Gamma, E., Johnson, R., Vlissides, J. and Helm, R. (1995) *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley.
- Grand, M. (1998) *Patterns in Java, Volume 1*, Wiley.
- Grand, M. (1999) *Patterns in Java, Volume 2*, Wiley.

Acknowledgements

Grateful acknowledgement is made to the following sources.

Figures

Figure 30: © Sun Microsystems

Index

A

Application controller pattern 19

application layer 8

architectural patterns 6

architectural styles 6

architectural views 6

architecture specialisation 14

architecture specification 14

C

component architectures 37

component collection 14

component infrastructures 37

component selection and specialisation 14

containers 38

create 27

D

data source layer 8

deployment view 6

design patterns 17

discoverable 42

domain analysis 14

domain layer 8

domain model 14

E

enterprise beans 34

Enterprise JavaBeans (EJB) 34

Enterprise JavaBeans container 34

enterprise systems 7

F

Factory method 27

Factory pattern 27

find, bind and invoke cycle 43

frameworks 11

functional (or logical) view 6

I

idioms 17

integration and release 15

interfaces 24

interoperability 41

L

layers 9

Layers pattern 6

location transparency 41

M

message-driven beans 35

Model–View–Controller pattern 18

O

Observer pattern 22

P

packages 9

presentation layer 8

process view 6

product line 13

product-line initiation 14

R

realisation 25

reference architectures 14

S

service-oriented architecture (SOA) 41

services 41

session beans 35

Singleton pattern 25

software architectures 6

specific-requirements capture 14

T

technical services layer 8

tiers 9

